Stepping back and thinking about something else, briefly

Does anyone here know Hubble's Law? (Without googling it or sneaking a peak at later slides)

Edwin Hubble

Astronomer who made critical contributions to our understanding of the universe. Moved to nearby Wheaton when he was 11, and attended University of Chicago

https://en.wikipedia.org/wiki/Edwin_Hubble

Edwin Hubble and his famous paper

https://www.pnas.org/content/pnas/15/3/168.full.pdf

A RELATION BETWEEN DISTANCE AND RADIAL VELOCITY AMONG EXTRA-GALACTIC NEBULAE

By Edwin Hubble

Mount Wilson Observatory, Carnegie Institution of Washington

Communicated January 17, 1929

https://en.wikipedia.org/wiki/Triangulum_Galaxy

Triangulum Galaxy (NGC 598)

TABLE 1

NEBULAE WHOSE DISTANCES HAVE BEEN ESTIMATED FROM STARS INVOLVED OR FROM MEAN LUMINOSITIES IN A CLUSTER

OBJECT	m_s	r	v	m_t	M_t
S. Mag.	..	0.032	+ 170	1.5	−16.0
L. Mag.	..	0.034	+ 290	0.5	17.2
N. G. C. 6822	..	0.214	− 130	9.0	12.7
598	..	0.263	− 70	7.0	15.1
221	..	0.275	− 185	8.8	13.4
224	..	0.275	− 220	5.0	17.2
5457	17.0	0.45	+ 200	9.9	13.3
4736	17.3	0.5	+ 290	8.4	15.1
5194	17.3	0.5	+ 270	7.4	16.1
4449	17.8	0.63	+ 200	9.5	14.5
4214	18.3	0.8	+ 300	11.3	13.2
3031	18.5	0.9	− 30	8.3	16.4
3627	18.5	0.9	+ 650	9.1	15.7
4826	18.5	0.9	+ 150	9.0	15.7
5236	18.5	0.9	+ 500	10.4	14.4
1068	18.7	1.0	+ 920	9.1	15.9
5055	19.0	1.1	+ 450	9.6	15.6
7331	19.0	1.1	+ 500	10.4	14.8
4258	19.5	1.4	+ 500	8.7	17.0
4151	20.0	1.7	+ 960	12.0	14.2
4382	..	2.0	+ 500	10.0	16.5
4472	..	2.0	+ 850	8.8	17.7
4486	..	2.0	+ 800	9.7	16.8
4649	..	2.0	+1090	9.5	17.0
Mean					−15.5

m_s = photographic magnitude of brightest stars involved.
r = distance in units of 10^6 parsecs. The first two are Shapley's values.
v = measured velocities in km./sec. N. G. C. 6822, 221, 224 and 5457 are recent determinations by Humason.
m_t = Holetschek's visual magnitude as corrected by Hopmann. The first three objects were not measured by Holetschek, and the values of m_t represent estimates by the author based upon such data as are available.
M_t = total visual absolute magnitude computed from m_t and r.

Edwin Hubble and his famous paper

https://www.pnas.org/content/pnas/15/3/168.full.pdf

A RELATION BETWEEN DISTANCE AND RADIAL VELOCITY AMONG EXTRA-GALACTIC NEBULAE

By Edwin Hubble

Mount Wilson Observatory, Carnegie Institution of Washington

Communicated January 17, 1929

1 pc = 3.1 x 10^{16} m
Positive v implies away from us, negative toward us

TABLE 1

NEBULAE WHOSE DISTANCES HAVE BEEN ESTIMATED FROM STARS INVOLVED OR FROM MEAN LUMINOSITIES IN A CLUSTER

OBJECT	m_s	r	v	m_t	M_t
S. Mag.	..	0.032	+ 170	1.5	−16.0
L. Mag.	..	0.034	+ 290	0.5	17.2
N. G. C. 6822	..	0.214	− 130	9.0	12.7
598	..	0.263	− 70	7.0	15.1
221	..	0.275	− 185	8.8	13.4
224	..	0.275	− 220	5.0	17.2
5457	17.0	0.45	+ 200	9.9	13.3
4736	17.3	0.5	+ 290	8.4	15.1
5194	17.3	0.5	+ 270	7.4	16.1
4449	17.8	0.63	+ 200	9.5	14.5
4214	18.3	0.8	+ 300	11.3	13.2
3031	18.5	0.9	− 30	8.3	16.4
3627	18.5	0.9	+ 650	9.1	15.7
4826	18.5	0.9	+ 150	9.0	15.7
5236	18.5	0.9	+ 500	10.4	14.4
1068	18.7	1.0	+ 920	9.1	15.9
5055	19.0	1.1	+ 450	9.6	15.6
7331	19.0	1.1	+ 500	10.4	14.8
4258	19.5	1.4	+ 500	8.7	17.0
4151	20.0	1.7	+ 960	12.0	14.2
4382	..	2.0	+ 500	10.0	16.5
4472	..	2.0	+ 850	8.8	17.7
4486	..	2.0	+ 800	9.7	16.8
4649	..	2.0	+1090	9.5	17.0
Mean					−15.5

m_s = photographic magnitude of brightest stars involved.
r = distance in units of 10^6 parsecs. The first two are Shapley's values.
v = measured velocities in km./sec. N. G. C. 6822, 221, 224 and 5457 are recent determinations by Humason.
m_t = Holetschek's visual magnitude as corrected by Hopmann. The first three objects were not measured by Holetschek, and the values of m_t represent estimates by the author based upon such data as are available.
M_t = total visual absolute magnitude computed from m_t and r.

Triangulum Galaxy (NGC 598)

What did Hubble find?

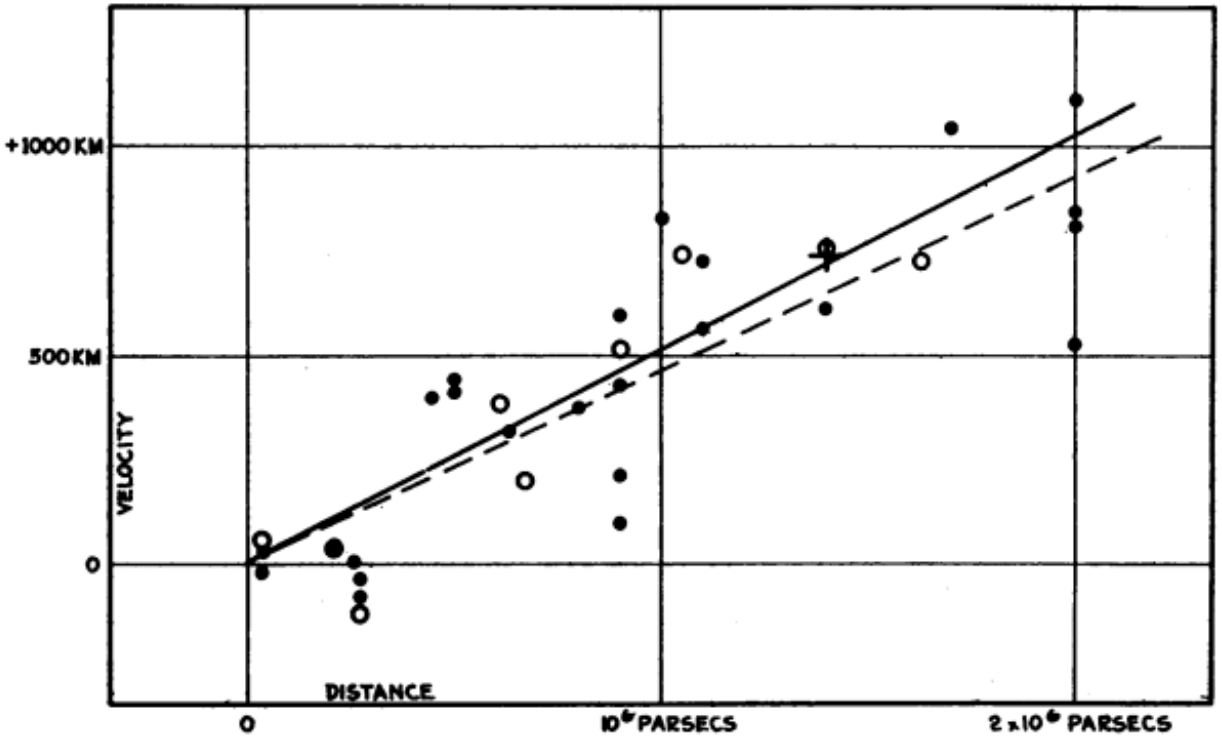


FIGURE 1

Velocity-Distance Relation among Extra-Galactic Nebulae.

Radial velocities, corrected for solar motion, are plotted against distances estimated from involved stars and mean luminosities of nebulae in a cluster. The black discs and full line represent the solution for solar motion using the nebulae individually; the circles and broken line represent the solution combining the nebulae into groups; the cross represents the mean velocity corresponding to the mean distance of 22 nebulae whose distances could not be estimated individually.

Clear linear relationship! How does one fit such curves, though?

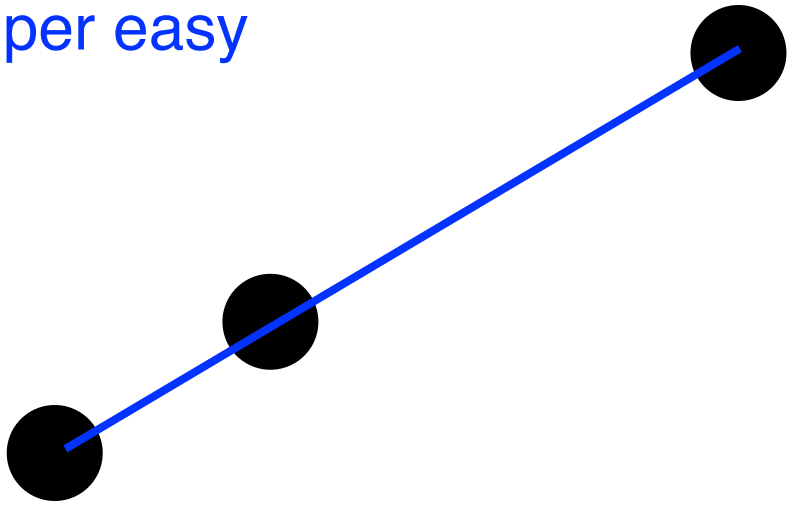
Well, that doesn't help, we can't fit a straight line to 0 or 1 point!

Linear fits

Well, with 2 points, this is easy

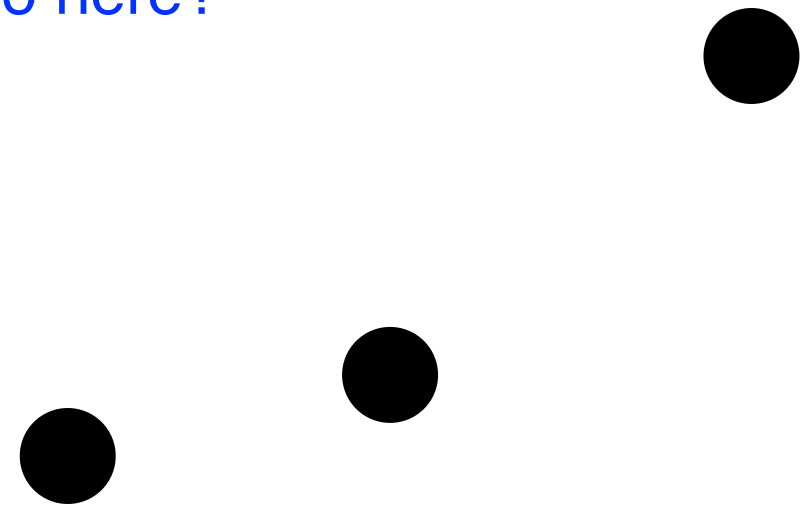
Linear fits

IF the 3 points line up and are colinear, this is also super easy



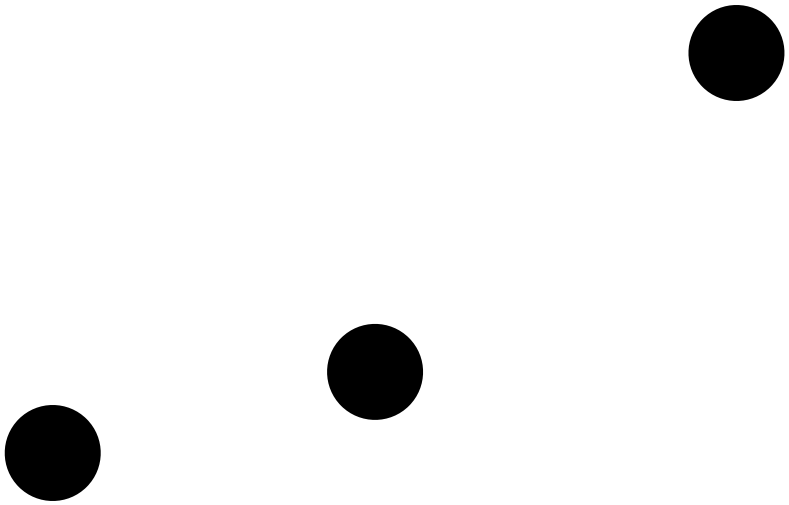
Linear fits

Uh oh, what do we do here?



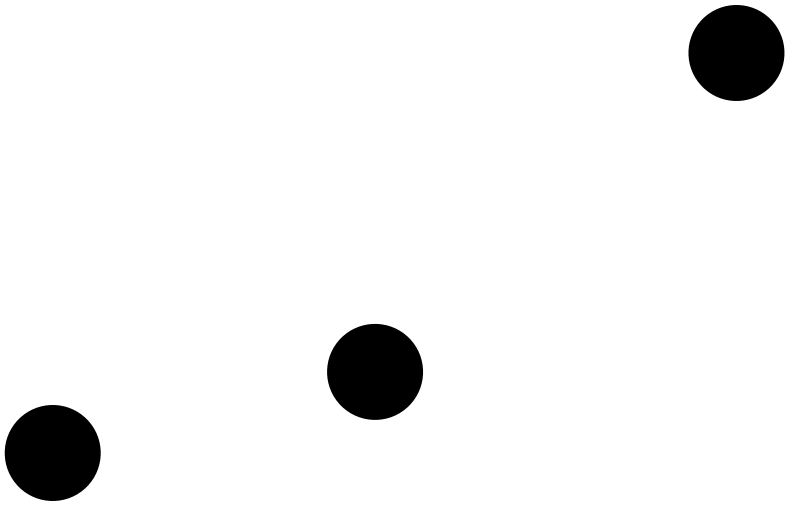
Well, there are a few reasons why our points might not all be colinear

Linear fits



Maybe each point is the result of statistical sampling. We repeat some measurement/sampling over and over; if so, this is easy to estimate and understand

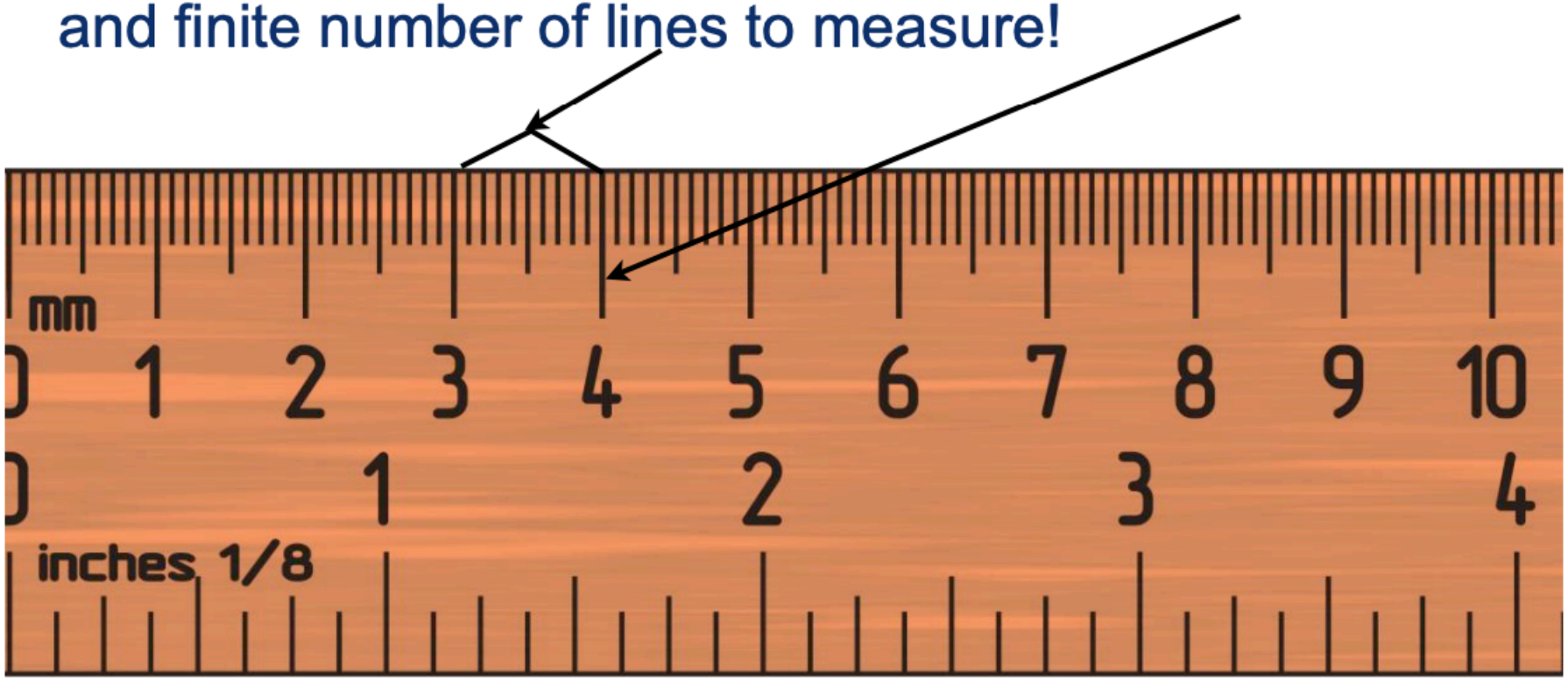
Linear fits



We can also have systematic uncertainties (maybe our ruler is not correct, maybe our stop watch is wrong, maybe there is human error, or otherwise). This is trickier.

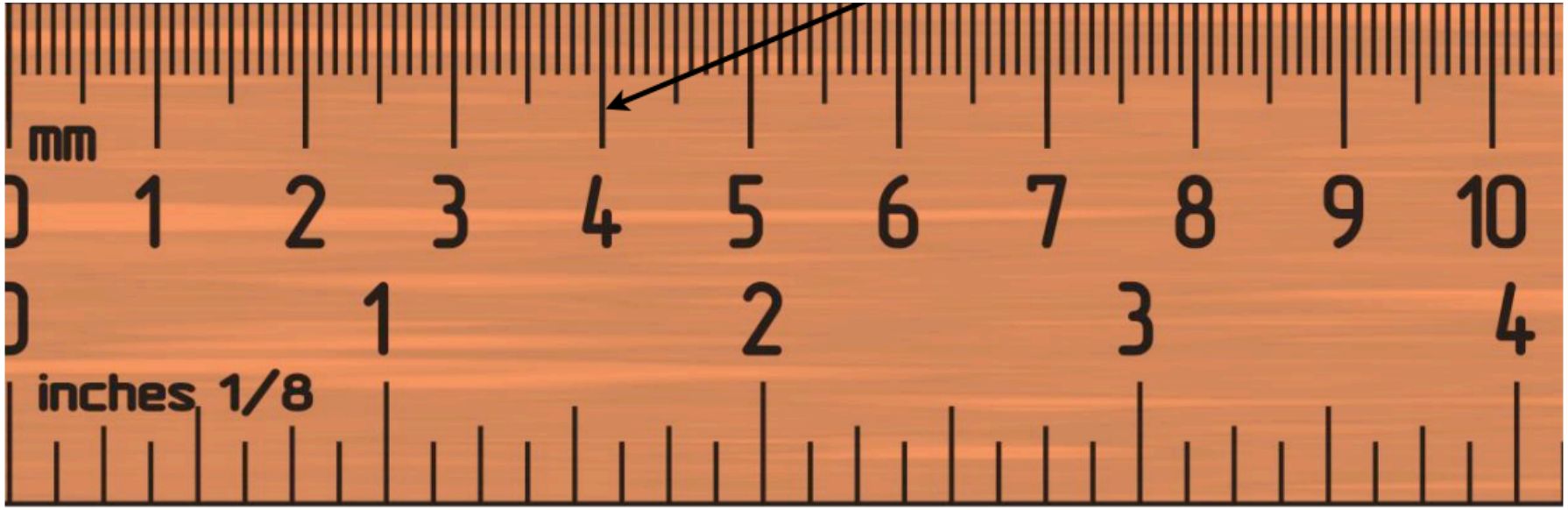
On uncertainties — Nice summary from Sal

- Example : measuring distance with a ruler
  - Systematic limitation : ruler has finite width of the lines, and finite number of lines to measure!



  - Statistical variation : you can try to repeat the same measurement over and over to get a better estimate of the "true" value

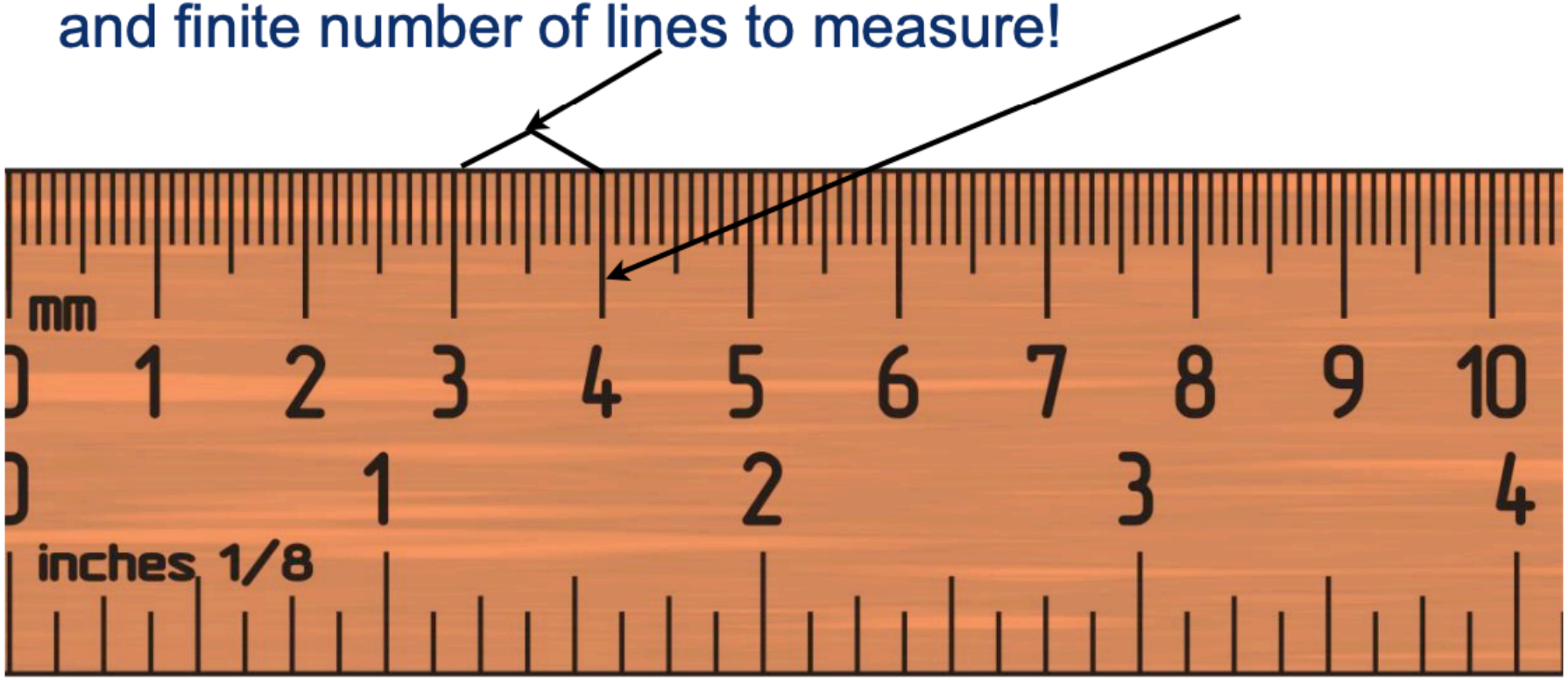
Repeat more measurements and this uncertainty shrinks!



– Statistical variation : you can try to repeat the same measurement over and over to get a better estimate of the "true" value

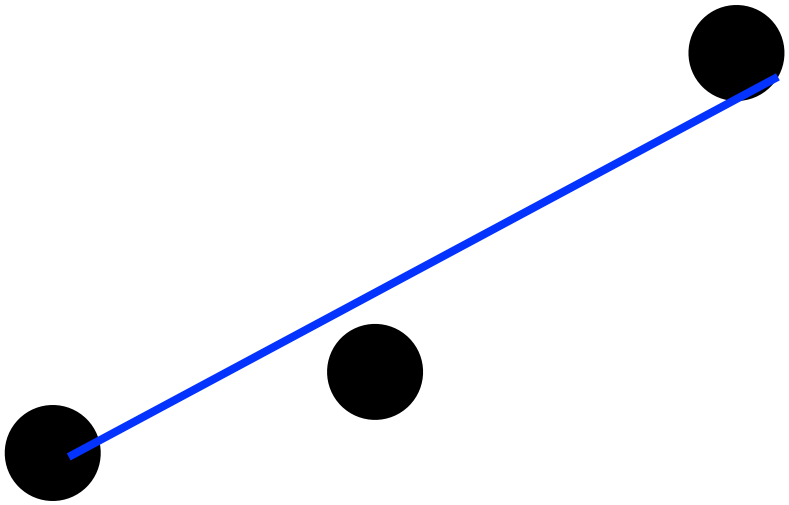
On uncertainties — Nice summary from Sal

- Example : measuring distance with a ruler
  - Systematic limitation : ruler has finite width of the lines, and finite number of lines to measure!



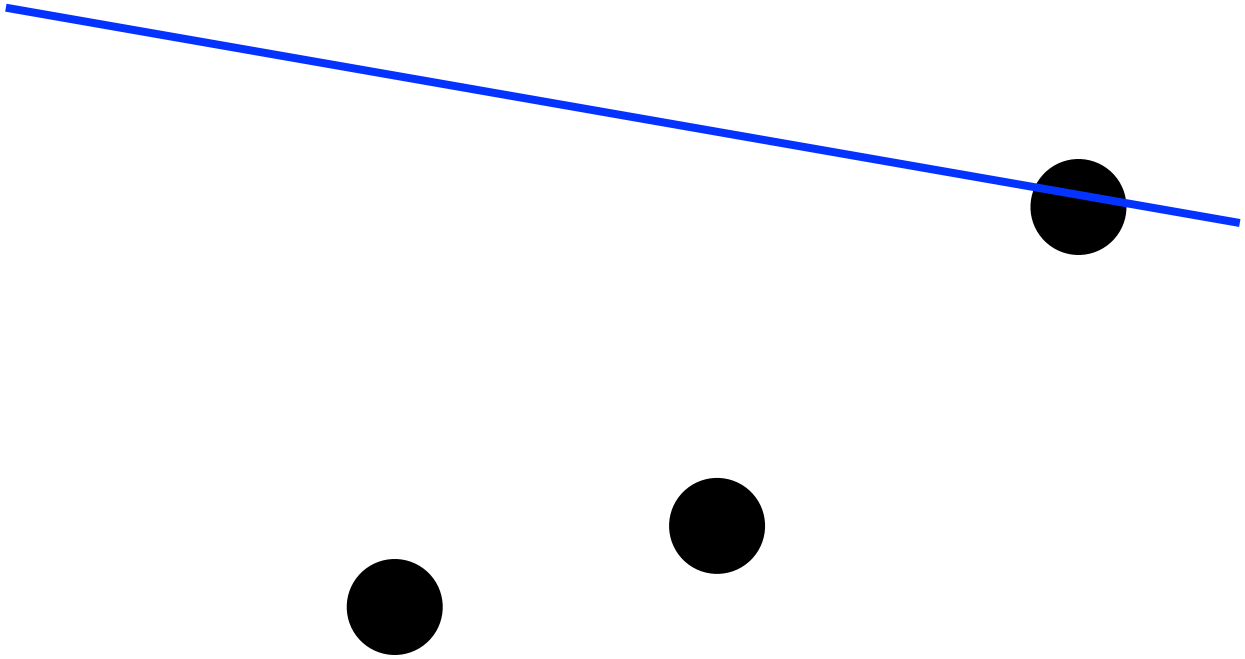
Unfortunately, more trials doesn't help with this. And it's harder to estimate!

Linear fits



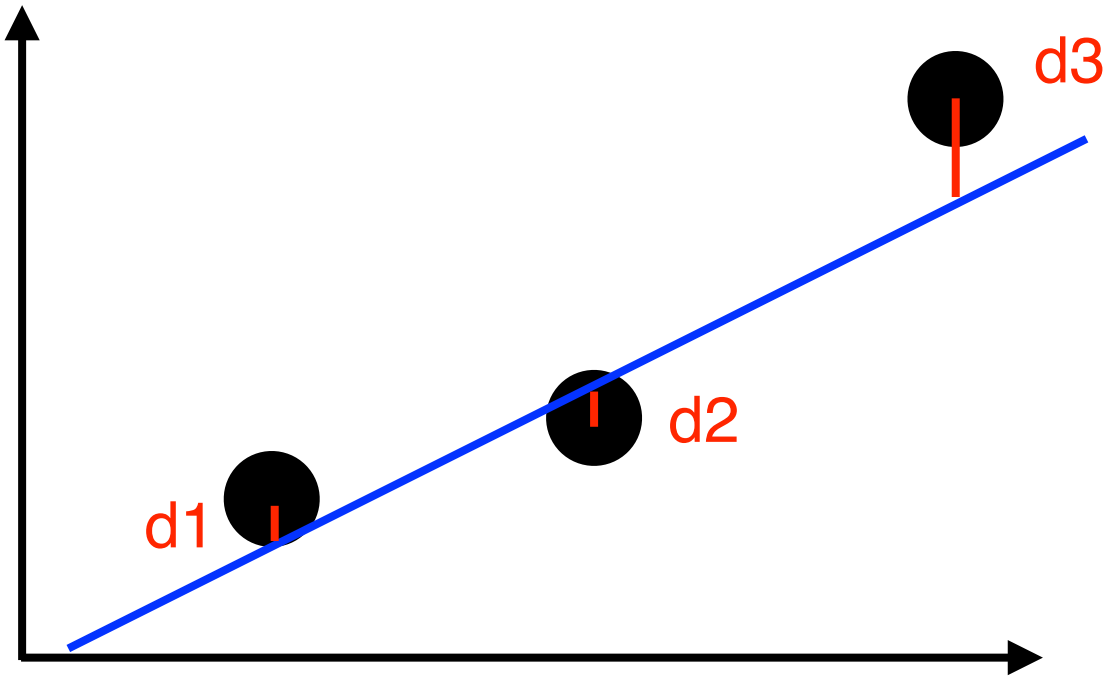
This looks like a reasonable guess and a good fit (of course, we will be more mathematically rigorous shortly)

How to do our linear fits?



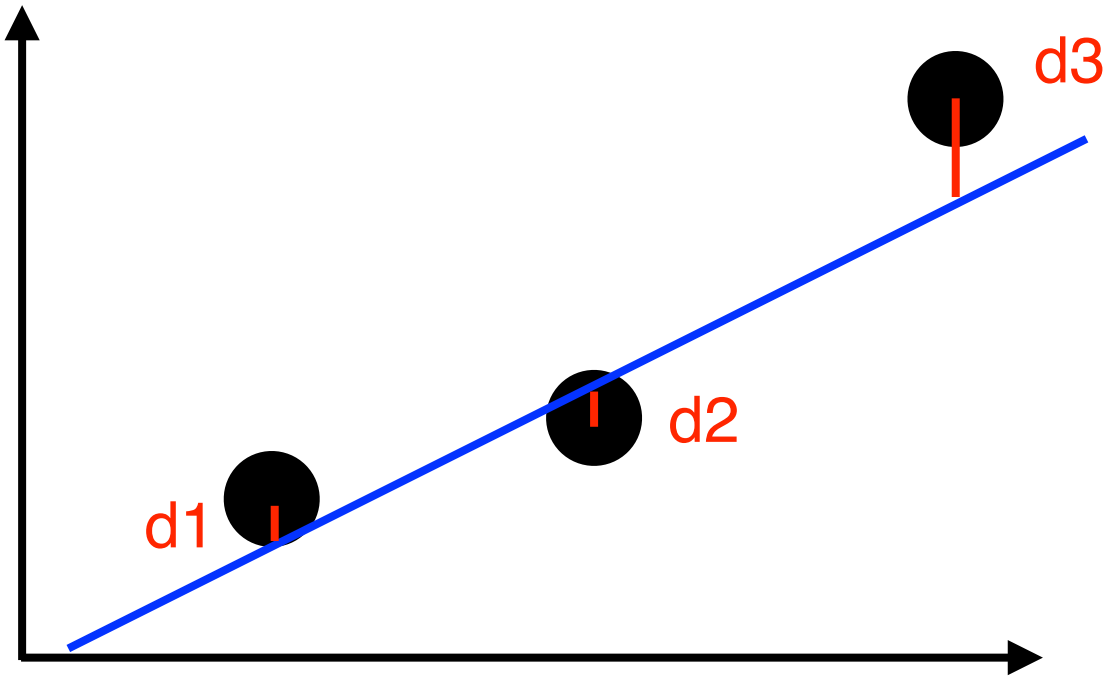
This is clearly not a good fit!

Why does the first line/fit look better?



Assume the errors are all the same (this assumption will be relaxed next). Our straight line is given by $y(x) = a+bx$, so that our prediction for point x_i is $y_{pred,i} = a+bx_i$ and then the distance between this and the measured value y_i is given by $y_i - y_{pred,i} = y_i - a- bx_i$

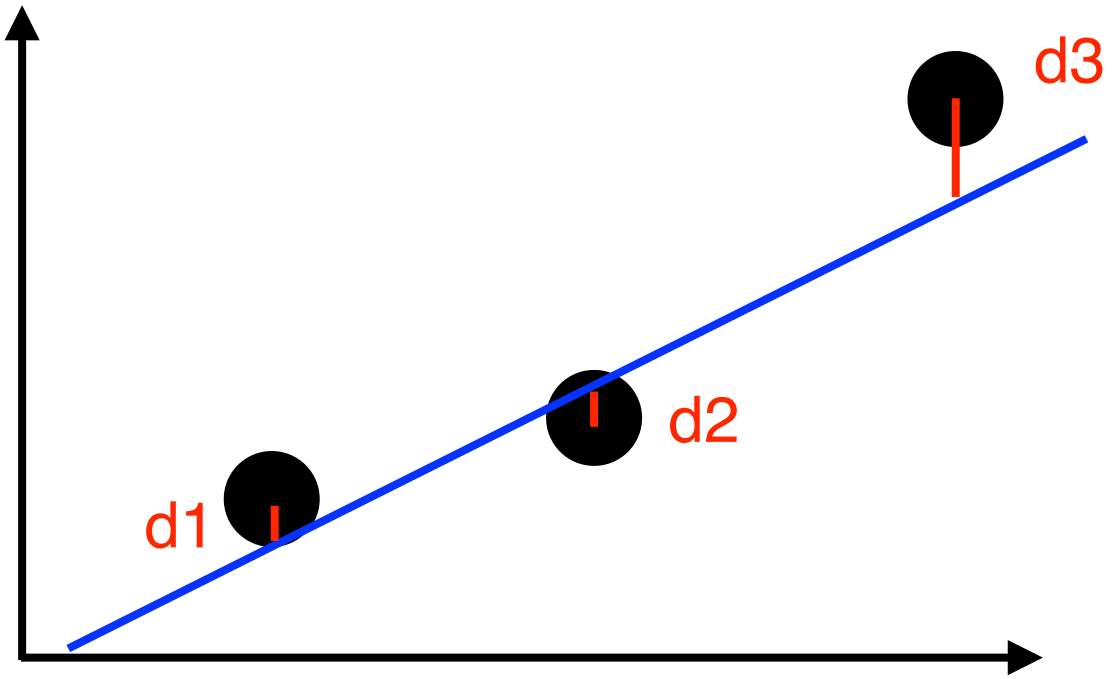
Why does the first line/fit look better?



The distance between the observed and measured value for point i is y_i - a- bx_i. Our total "distance" squared is then:

$$f(a,b) = \sum_i (y_i - a - bx_i)^2$$

How to get best-fit parameters



We want to **minimize** the following quantity:

$$f(a,b) = \sum_i (y_i - a - bx_i)^2$$

How to get best-fit parameters

We want to **minimize** the following quantity:

$$f(a,b) = \sum_i (y_i - a - bx_i)^2$$

$$\frac{\partial f}{\partial a} = -2\sum_i (y_i - a - bx_i)$$

$$\frac{\partial f}{\partial b} = -2\sum_i x_i(y_i - a - bx_i)$$

Two equations, two unknowns (a,b)

How to get best-fit parameters

$$\frac{\partial f}{\partial a} = -2\sum_i (y_i - a - bx_i) = 0$$

$$\frac{\partial f}{\partial b} = -2\sum_i x_i(y_i - a - bx_i) = 0$$

$$\sum_i (y_i - a - bx_i) = 0$$

$$\sum_i x_i(y_i - a - bx_i) = 0$$

$$\sum_i y_i - Na - b\sum x_i = 0$$

$$\sum_i x_i y_i - a\sum x_i - b\sum x_i^2 = 0$$

How to get best-fit parameters

$$\sum_i y_i - Na - b\sum x_i = 0$$

$$\sum_i x_i y_i - a\sum x_i - b\sum x_i^2 = 0$$

$$a = \frac{1}{N}\sum y_i - b\frac{1}{N}\sum x_i$$

$$a = \frac{1}{\sum x_i}\sum x_i y_i - b\frac{1}{\sum x_i}\sum x_i^2$$

How to get best-fit parameters

$$a = \frac{1}{N}\sum y_i - b\frac{1}{N}\sum x_i$$

$$a = \frac{1}{\sum x_i}\sum x_i y_i - b\frac{1}{\sum x_i}\sum x_i^2$$

$$s_y/N - (b/N)s_x = (s_{xy}/s_x) - b(s_{xx}/s_x)$$

$$b(s_{xx}/s_x - s_x/N) = s_{xy}/s_x - s_y/N$$

$$b(Ns_{xx} - s_x^2) = Ns_{xy} - s_x s_y$$

$$b = (Ns_{xy} - s_x s_y)/(Ns_{xx} - s_x * s_x)$$

How to get best-fit parameters

$$a = \frac{1}{N}\sum y_i - b\frac{1}{N}\sum x_i$$

$$a = \frac{1}{\sum x_i}\sum x_i y_i - b\frac{1}{\sum x_i}\sum x_i^2$$

$$s_y/N - a = (b/N)s_x$$

$$s_{xy}/s_x - a = (b/s_x)s_{xx}$$

$$b = s_y/s_x - aN/s_x = s_{xy}/s_{xx} - as_x/s_{xx}$$

$$s_y/s_x - s_{xy}/s_{xx} = a(N/s_x - s_x/s_{xx})$$

$$a = (s_y/s_x - s_{xy}/s_{xx})/(N/s_x - s_x/s_{xx})$$

$$a = (s_y * s_{xx} - s_{xy} * s_x)/(N * s_{xx} - s_x^2)$$

How to get best-fit parameters

$$a = (s_y * s_{xx} - s_{xy} * s_x)/(N * s_{xx} - s_x^2)$$

$$b = (N * s_{xy} - s_x * s_y)/(N * s_{xx} - s_x^2)$$

Just a few simple sums to
calculate, pretty straightforward!

Is that the full picture?

We also want to know three additional, very important quantities:

1) The uncertainty on a
2) The uncertainty on b
3) The uncertainty per degree of freedom of the fit (aka the goodness of the fit). For our linear fit we have two constraints so the variance of the fit, or goodness of fit, is:

$$\sigma^2 = \frac{1}{n-2}\sum (y_i - y(x_i))^2$$

And what do we do if the uncertainty on each point is not equal? Minimize the chi2 instead!

$$\chi^2(a,b) = \sum \left(\frac{y_i - a - bx_i}{\sigma_i} \right)^2$$

More complicated fits

And what do we do if the uncertainty on each point is not equal? Minimize the chi2 instead!

$$\chi^2(a,b) = \sum \left(\frac{y_i - a - bx_i}{\sigma_i} \right)^2$$

If the uncertainties on each point are constant, then minimizing the above gives our previous result. If not, it provides more weight in the fit to points with smaller error, and less weight in the fit to points with larger error

Using point-by-point errors

Uncertainties on parameters!

$$b = \frac{1}{S_{tt}} \sum_{i=0}^{n-1} \frac{t_i y_i}{\sigma_i} \,, \qquad a = \frac{S_y - S_x b}{S} \qquad \sigma_a^2 = \frac{1}{S}\left(1 + \frac{S_x^2}{S S_{tt}}\right) \,, \qquad \sigma_b^2 = \frac{1}{S_{tt}}$$

with

$$t_i = \frac{1}{\sigma_i}\left(x_i - \frac{S_x}{S}\right) \,, \qquad S_{tt} = \sum_{i=0}^{n-1} t_i^2 \,,$$

$$S = \sum_{i=0}^{n-1} \frac{1}{\sigma_i^2} \,, \qquad S_x = \sum_{i=0}^{n-1} \frac{x_i}{\sigma_i^2} \,, \qquad S_y = \sum_{i=0}^{n-1} \frac{y_i}{\sigma_i^2}$$

And goodness of fit :

$$\chi^2(a,b)/ndof = \frac{1}{n-2} \sum \left(\frac{y_i - a - b x_i}{\sigma_i}\right)^2$$

Using point-by-point errors

And goodness of fit :

$$\chi^2(a,b)/ndof = \frac{1}{n-2}\sum\left(\frac{y_i - a - bx_i}{\sigma_i}\right)^2$$

If our data really follow a linear distribution AND we have estimated our errors correctly, we should get chi2/ndf values ~1. Either way, we can use the chi2 and n to calculate a p-value indicating our goodness of fit

Other types of fits

How do we fit other curves to our data? One obvious thing to do is to transform the data so that it is a straight line! For example:

$$y = Ae^{Bx} \rightarrow \ln y = \ln A + Bx$$

So if we fit ln(y) as a function of x, we can use the same tools as before, just remembering that B is the slope and our constant is ln(A)

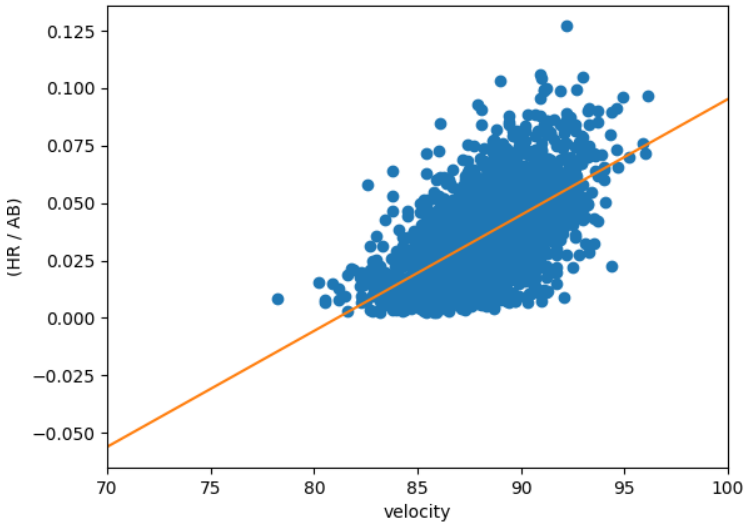
Uncertainties with alternative fits

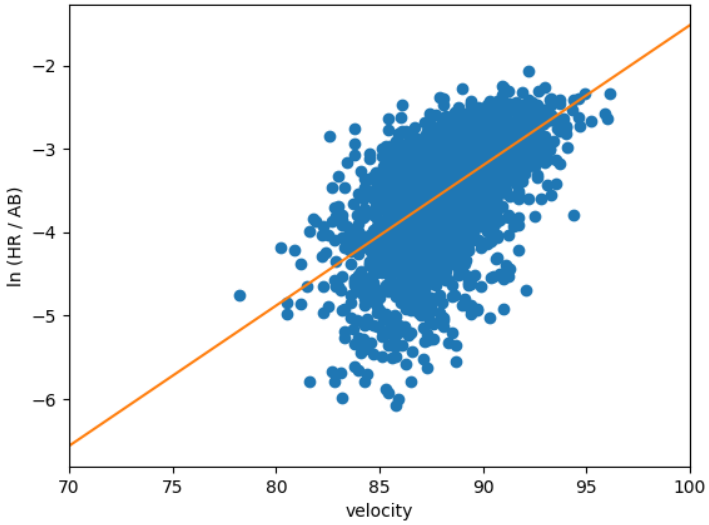
$$y = Ae^{Bx} \rightarrow \ln y = \ln A + Bx$$

$$y' = f(y) \rightarrow \sigma_{y'}^2 = \left(\frac{\partial f}{\partial y}\right)^2 \sigma_y^2$$

$$y' = \ln y = \ln A + Bx \rightarrow \sigma_{y'} = \frac{\sigma_y}{|y|}$$

Example from baseball

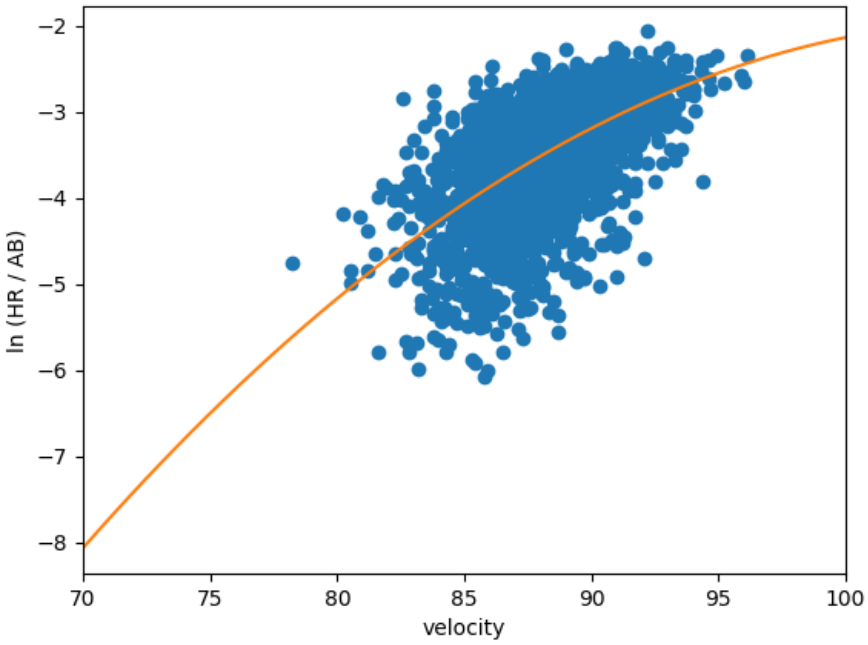




The harder you hit the ball, the more home runs you should hit. Is a linear fit really right? Exponential may be better (need to do goodness of fit checks to decide this!)

Also, quick Q: What is wrong with my plots? (Good check for you to make…)

Example from baseball



Try fitting a 2nd order polynomial to the log (equivalent to $y=Ae^{Bx+Cx**2}$

Aside from goodness of fit tests, how might you determine if this is a better fit?

How to fit a more general function?

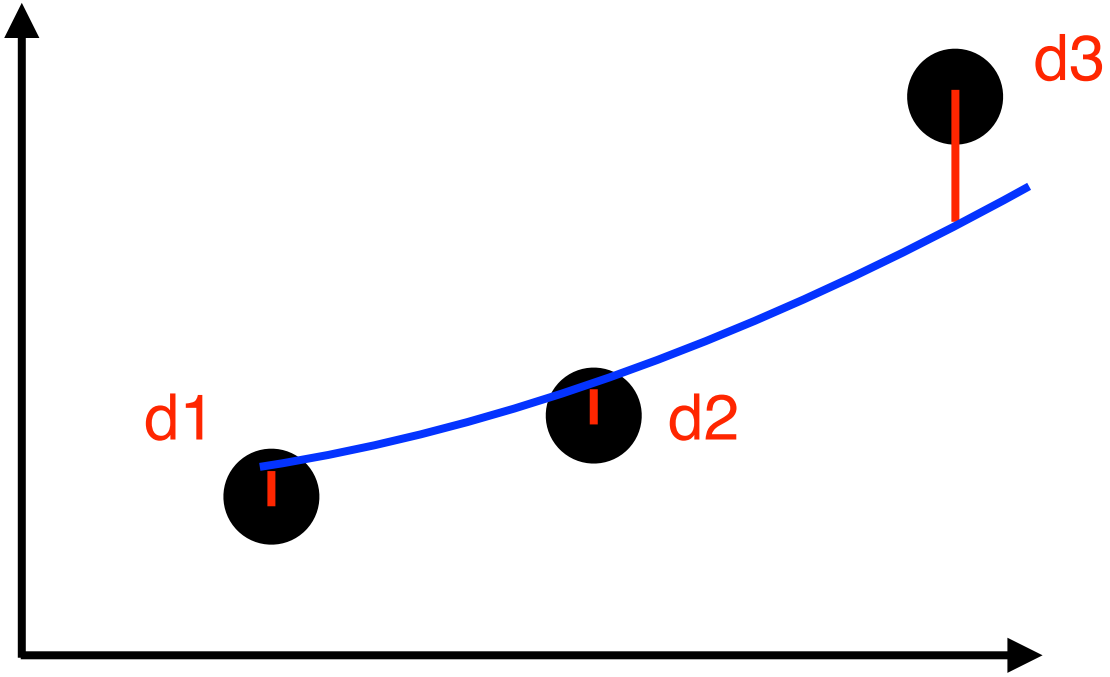
$$y = y(x; \vec{p})$$

y is a function of x, with vector of parameters **p**

$$\chi^2(\vec{p}) = \sum_i \left(\frac{y_i - y(x; \vec{p})}{\sigma_i} \right)^2$$

Minimize this for **p**

Still minimizing the total distance between prediction and observation



What other kinds of fits can we do?

If we normalize our fit function, then for any set of parameters we know the probability that the fit function gave any one data point - it's the function itself! Call that the probability of the data point

Given a set of parameters for our normalized fit function, we can multiply the probabilities for each of our data points to give the joint probability (our likelihood, L) of all data!

Maximum likelihood fits

Multiplying probabilities is OK, but we end up with very large or small numbers. And it's hard to maximize the joint probability in many cases. To make this easier we can take the log of the likelihood and maximize that instead probability with respect to all parameters (the natural log, ln, is a monotonically increasing function, so this still works!)

This is a cool way to answer the question: "For which parameter value does the observed data have the biggest probability?"

Let's give a concrete example

Fit data to function $f(x) = kx^2e^{cx}$, where k is a normalization term . Imagine that c is negative and our data of N points goes from 0 to infinity (in practice we will cut this off at some large value). The constant k is not a free parameter but is really k(c), as we must set k to normalize our fit function

Then $L = \Pi_i kx^2e^{cx}$, and $\ln L = \ln \Pi kx_i^2e^{cx_i}$

$\ln L = [\Sigma_i \ln(k) + \ln(x_i^2) + cx_i] = N \ln k + 2\Sigma_i ln(x_i) + c\Sigma_i x_i$

If we want to maximize lnL we can also maximize ln L/N

ln L/N = ln k + 2<ln(x)> + c<x>. Looks simple… but what is k?

We can estimate k(c) numerically for a given c, but thankfully Wikipedia gives us this analytical answer!

$$\int xe^{cx}\,dx = e^{cx}\left(\frac{cx-1}{c^2}\right) \qquad \text{for } c \neq 0;$$

$$\int x^2e^{cx}\,dx = e^{cx}\left(\frac{x^2}{c} - \frac{2x}{c^2} + \frac{2}{c^3}\right)$$

$$\int x^ne^{cx}\,dx = \frac{1}{c}x^ne^{cx} - \frac{n}{c}\int x^{n-1}e^{cx}\,dx$$

Let's normalize our fit function

$$\int_0^\infty kx^2e^{cx} = 0 - ke^0\left(0/c - 0/c^2 + 2/c^3\right) = -2k/c^3 = 1,$$

So k = -c³/2 and $f(x) = -\frac{c^3}{2}x^2e^{cx}$. Then:

ln L/N = ln (-c³/2) + 2<ln(x)> + c<x>.

$$\int xe^{cx}\,dx = e^{cx}\left(\frac{cx-1}{c^2}\right) \qquad \text{for } c \neq 0;$$

$$\int x^2e^{cx}\,dx = e^{cx}\left(\frac{x^2}{c} - \frac{2x}{c^2} + \frac{2}{c^3}\right)$$

$$\int x^ne^{cx}\,dx = \frac{1}{c}x^ne^{cx} - \frac{n}{c}\int x^{n-1}e^{cx}\,dx$$

Let's maximize ln(L/N)

$$\int_0^\infty kx^2e^{cx} = 0 - ke^0\left(0/c - 0/c^2 + 2/c^3\right) = -2k/c^3 = 1,$$

So k = -c³/2 and $f(x) = -\frac{c^3}{2}x^2e^{cx}$. Then:

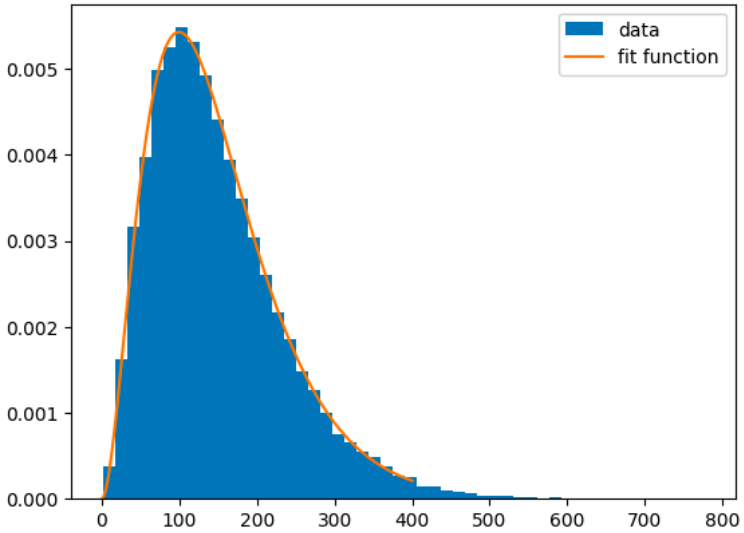
ln L/N = ln(-c³/2) + 2<ln(x)> + c<x>. Given a set of data, the only free parameter is c. How to maximize ln(L/N) for this data? The derivative with respect to c but must be zero!

$\frac{\partial(lnL/N)}{\partial c}$ = 3/c + <x> = 0 → c = -3/<x>, which is good, it's negative as we wanted! Is this going to minimize or maximize the probability? Take the second derivative:

$\frac{\partial^2(lnL/N)}{\partial c^2}$ = -3/c², which is negative, so it's a maximum!

Let's try it out


```
import matplotlib.pyplot as plt
import numpy as np
from google.colab import files
uploaded = files.upload()### first fit data
points = np.genfromtxt('fitdata.txt')
N = points.size
average = np.sum(points)/N
c_estimated = -3/average
print("c estimated = ",c_estimated)
min=0
max=400

def func_fixed(xs):
    return -c_estimated*c_estimated*c_estimated/(2)*xs*xs*np.exp(c_estimated*xs)


plt.hist(points,bins = 50, density = True,label="data")
plt.plot(np.linspace(min,max,1000), np.frompyfunc(func_fixed, 1, 1)(np.linspace(min,max,1000)),label="fit function")
plt.legend()
plt.show()
```



Cool! We didn't even need to do any work. Sometimes we can't get an analytical answer like this but then we can scan across parameters and find the values that maximize the log-likelihood (using some techniques from this and other chapters)

Let's try another one

Fit data to function $f(x) = kx^2 e^{-(ax^2+bx)}$, where k is a normalization term and a > 0 . Imagine that data of N points goes from -infinity to infinity (in practice we will cut this off at some values). The constant k is not a free parameter but is really k(a,b), as we must set k to normalize our fit function

This is a different way to write a Gaussian centered not at zero!

Then $L = \Pi_i k x_i^2 e^{-(ax_i^2+bx_i)}$, and $\ln L = \ln \Pi_i k x_i^2 e^{-(ax_i^2+bx_i)}$

$$\ln L = \Sigma_i \ln(k) + \ln(x_i^2) - ax_i^2 - bx_i =$$
$$N \ln k + 2N < \ln(x) > - aN < x^2 > - bN < x >$$

If we want to maximize lnL we can also maximize ln L/N
ln L/N = ln k + 2<ln(x)> -a<x²> - b<x>. What is k?

Let's normalize our second fit function

$$\int_0^\infty kx^2 e^{-(ax^2+bx)} = k\sqrt{\frac{\pi}{a}}e^{b^2/(4a)}$$

So k = $\sqrt{\frac{a}{\pi}}e^{-b^2/(4a)}$ and $f(x) = \sqrt{\frac{a}{\pi}}e^{-b^2/(4a)}x^2 e^{-(ax^2+bx)}$

We had:

ln L/N = ln k + 2<ln(x)> -a<x²> - b<x>

So ln L/N = 0.5(ln a - lnπ) + $\frac{-b^2}{4a}$ + 2<ln(x)> -a<x²> - b<x>

How do we maximize this? BOTH partial derivatives should be zero!

Let's normalize our second fit function

$$\ln L/N = 0.5(\ln a - \ln\pi) + \frac{-b^2}{4a} + 2\langle \ln(x)\rangle - a\langle x^2\rangle - b\langle x\rangle$$

$$\frac{\partial lnL/N}{\partial a} = 0.5/a + \frac{b^2}{4a^2} - \langle x^2\rangle = 0$$

$$\frac{\partial lnL/N}{\partial b} = \frac{-b}{2a} - \langle x\rangle = 0$$

Rewrite the first of this: $0.5a + b^2/4 - a^2\langle x^2\rangle = 0$

From the second: $b = -2a\langle x\rangle$

Plus this back in: $0.5a + a^2\langle x\rangle^2 - a^2\langle x^2\rangle = 0$. We don't want a=0

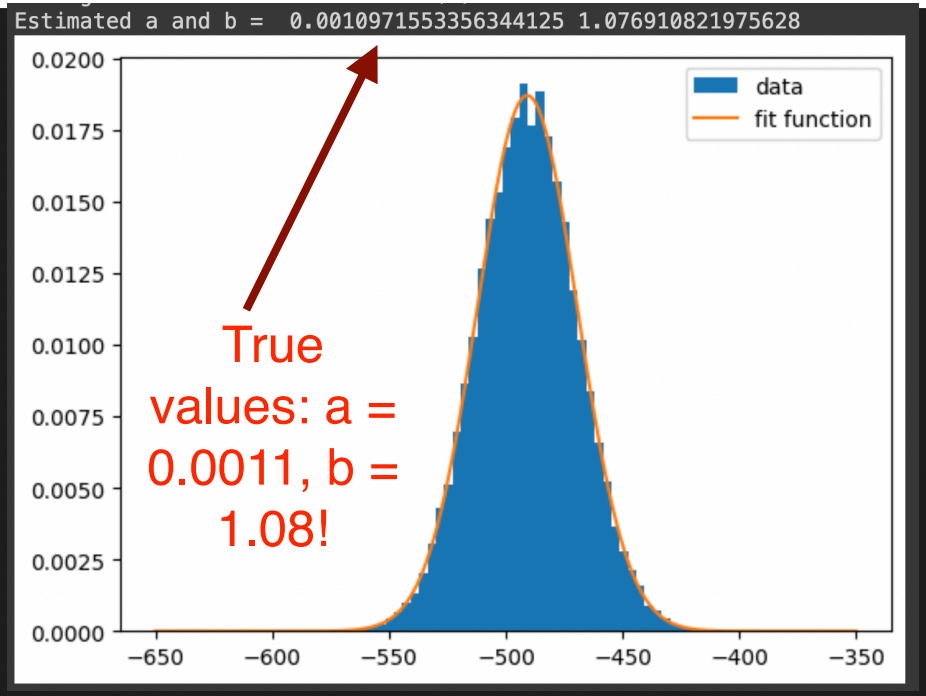
So $0.5 + a\langle x\rangle^2 - a\langle x^2\rangle = 0$, or **$a = 1./(2*(\langle x^2\rangle-\langle x\rangle^2))$**

And then **$b = -\langle x\rangle/(\langle x^2\rangle-\langle x\rangle^2))$**

Trying out our second fit


```
import matplotlib.pyplot as plt
import numpy as np
from google.colab import files

def func_fixed(xs):
    k = np.sqrt(np.pi/a)*np.exp(b*b/(4*a))
    return (1./k)*np.exp(-(a*xs*xs + b*xs))

uploaded = files.upload()### first fit data
points = np.genfromtxt('fitdata3.txt')

min=-650
max=-350

N = points.size
avgx = np.sum(points)/N
avgx2 = np.sum(points*points)/N
a = 1./(2*(avgx2 - avgx*avgx))
b = -avgx/(avgx2 - avgx*avgx)
print("Estimated a and b = ",thisa,thisb)

plt.hist(points,bins = 50, density = True,label="data")
plt.plot(np.linspace(min,max,1000), np.frompyfunc(func_fixed, 1, 1)(np.linspace(min,max,1000)),label="fit function")
plt.legend()
plt.show()
```

```
Estimated a and b =  0.0010971553356344125 1.076910821975628
```



It's great when we can do this analytically! We won't have time to try fits where that isn't possible, for a second semester of this course? :)

How the book approaches this

Imagine we want to fit f(x) = y=mx+b.

Assume each measured point y_i has the same measurement error, σ. The value of $y_i = y(x_i)$ we fit is not the true value but moves around from this true value due to the measurement error, σ.

In that case, we can write y=mx+b+η, where η is the shift on y from the true value.

If we have Gaussian statistics

$$P(\eta \mid \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\eta^2/(2\sigma^2)}$$, but we can rewrite this as

$$P(\eta \mid \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-(y-mx-b)^2/(2\sigma^2)}$$

So rewriting this

$$P(y \mid x, m, b, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-(y-mx-b)^2/(2\sigma^2)}$$

Now let's do a maximum likelihood fit on this!

How the book approaches this

$$P(y\,|\,x,m,b,\sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-(y-mx-b)^2/(2\sigma^2)}$$

We have N sets of measurements, x_i and y_i

$$P(y1..y_N\,|\,x_1 \ldots x_N,m,b,\sigma) = \left(\frac{1}{\sqrt{2\pi\sigma^2}}\right)^N \Pi_i e^{-(y_i-mx_i-b)^2/(2\sigma^2)}$$

Likelihood = $P(y1..y_N\,|\,x1...x_N,m,b,\sigma) = \sqrt{2\pi\sigma^2}^{-N/2} \Pi_i e^{-(y_i-mx_i-b)^2/(2\sigma^2)}$

Log Likelihood = ln $\left[\sqrt{2\pi\sigma^2}^{-N/2} \Pi_i e^{-(y_i-mx_i-b)^2/(2\sigma^2)}\right]$

Log Likelihood = -N/2 ln $(\sqrt{2\pi\sigma^2}) - \frac{1}{2\sigma^2}\sum_i (y_i - mx_i - b)^2$

LL/N = -1/2 ln $\sqrt{2\pi\sigma^2}$

$$-\frac{1}{2\sigma^2}\left[< y^2 > + m^2 < x^2 > + b^2 - 2m < xy > - 2b < y > + 2mb < x > \right]$$

How the book approaches this

LL/N = -1/2 ln $\sqrt{2\pi\sigma^2}$

$$-\frac{1}{2\sigma^2}\left[< y^2 > + m^2 < x^2 > + b^2 - 2m < xy > - 2b < y > + 2mb < x > \right]$$

Now take our partial derivatives

$$\frac{\partial\, LL/N}{\partial m} = -\frac{1}{2\sigma^2}\left[2m < x^2 > - 2 < xy > + 2b < x > \right] = 0$$

$$\frac{\partial\, LL/N}{\partial b} = -\frac{1}{2\sigma^2}\left[2b - 2 < y > + 2m < x > \right] = 0$$

$$m < x^2 > - < xy > + b < x > = 0$$ From first partial derivative

$$b - < y > + m < x > = 0$$ From second partial derivative

How the book approaches this

$$m < x^2 > - < xy > + b < x > = 0$$

$$b - < y > + m < x > = 0$$ Rewrite this $$b = < y > - m < x >$$

Plug back into first equation

$$m < x^2 > - < xy > + (< y > - m < x >) < x > = 0$$

$$m(< x^2 > - < x >^2) - < xy > + < x > < y > = 0$$

$$m = \frac{< xy > - < x > < y >}{< x^2 > - < x >^2}$$

What we previously got!

How the book approaches this

$$m < x^2 > - < xy > + b < x > = 0$$

$$b - < y > + m < x > = 0$$ Rewrite this $$m = \frac{< y > - b}{< x >}$$

Plug back into first equation

$$\frac{< y > - b}{< x >} < x^2 > - < xy > + b < x > = 0$$

$$(< y > - b) < x^2 > - < xy > < x > + b < x >^2 = 0$$

$$b(< x >^2 - < x^2 >) = < xy > < x > - < x^2 > < y >$$

$$b = \frac{< xy > < x > - < x^2 > < y >}{(< x >^2 - < x^2 >)}$$ **What we previously got!**

How the book approaches this

Log Likelihood = -N/2 ln $(\sqrt{2\pi\sigma^2}) - \frac{1}{2\sigma^2}\sum_i (y_i - mx_i - b)^2$

Now take one more partial derivative

$$\frac{\partial\, LL/N}{\partial\sigma} = -\frac{N}{\sigma} + \frac{1}{\sigma^3}\sum_i (y_i - mx_i - b)^2 = 0$$

$$\frac{1}{\sigma^3}\sum_i (y_i - mx_i - b)^2 = \frac{N}{\sigma}$$

$$\frac{1}{N}\sum_i (y_i - mx_i - b)^2 = \sigma^2$$

Now we get our error on the measurements, cool!

Bayes

A subtle point about what we just did. We maximized

$$P(y \mid x, m, b, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-(y-mx-b)^2/(2\sigma^2)}$$

We maximized the probability of our observations given the parameters of the model. That is NOT really what we want. We want to maximize the probability of the model given the data. A subtle but important point. How do we deal with that?

Bayes: $P(A,B) = P(A)P(B \mid A)$ even if A and B are dependent on one another. Read this as "The joint probability of A and B is the probability of A times the probability of B given A".

BUT we can also write this as $P(A,B) = P(B)P(A \mid B)$, which is "The joint probability of A and B is the probability of B times the probability of A given B"

So we can equate these two $P(A,B) = P(B)P(A \mid B) = P(A)P(B \mid A)$ so

$$P(A \mid B) = \frac{P(A)}{P(B)} P(B \mid A)$$

Bayes with multiple variables

$$P(A \mid B, C) = \frac{P(A)}{P(B, C)} P(B, C \mid A)$$

Read as:
Probability of A given B and C is the probability of B and C given A times the ratio of probability A to the probability of B and C

$$P(A \mid B, C) = \frac{P(A)}{P(B)} P(B \mid A, C)$$

Read as:
Probability of A given B and C is the probability of B given A and C times the ratio of probability A to the probability of B

A great Bayes example

You get tested for a rare genetic disease that affects 1 out of every 100,000 people. You take a genetic test that correctly identifies 94% of the people who have the disease and 99% of the people who do not have the disease. The test comes back positive. Should you worry?

P(disease) = 1/100,000, so P(no disease) = 1-1/100,000
P(positive | disease) = 0.94
P(negative | no disease) = 0.99
The last one implies false positive rate P(positive | no disease) = 0.01

What is P(disease | positive)? Bayes tells us:
P(disease | positive) = P(positive | disease) * P(disease)/P(positive)

What is P(positive)?
P(positive) = P(positive | disease)P(disease) + P(positive | no disease)P(no disease)
P(positive) = (0.94)/100,000 + (0.01)(1-1/100,000) = 0.0100093

P(disease | positive) = 0.94 * (1/100,000)/(0.0100093) = 0.00094 = 0.094%. So even with a positive test result, you are unlikely to have this disease because it is so rare!

Other Bayes examples

A person uses his car 30% of the time, walks 30% of the time and rides the bus 40% of the time as he goes to work. He is late 10% of the time when he drives his car; he is late 5% of the time when he walks; and he is late 20% of the time he takes the bus.

P(car) = 0.3, P(walk) = 0.3, P(bus) = 0.4
P(late | car) = 0.1, P(late | walk) = 0.05, P(late | bus) = 0.2

What is the probability he took the bus if he was late?
P(bus | late) = P(bus)/P(late) * P(late | bus)

P(late) = P(late | car)P(car) + P(late | walk)P(walk) + P(late | bus)P(bus)
P(late) = 0.1(0.3) + 0.05(0.3) + 0.2(0.4) = 0.125

So P(bus | late) = 0.4/0.125 * 0.2 = 0.64 = 64%

What is the probability he walked if he is on time?
P(NOT late | car) = 0.9, P(NOT late | walk) = 0.95, P(NOT late | bus) = 0.8
P(walk | NOT late) = P(walk)/P(NOT late) * P(NOT late | walk)
P(NOT late) = P(NOT late | car)P(car) + P(NOT late | walk)P(walk) + P(NOT late | bus)P(bus)
P(NOT late) = 0.9*0.3 + 0.95*0.3 + 0.8*0.4 = 0.875
So P(walk | NOT late) = 0.3/0.875 * 0.95 = 0.33 = 33%

Back to our maximum likelihood

P(parameters | data) = P(parameters)/P(data) * P(data| parameters)

The distribution P(parameters) is our **prior**, ie our previous best estimate of the parameters. Maybe we have some prior information that tells us something about them. If not, we can say it's flat. And P(data) doesn't tell us anything so:

P(parameters | data) = P(data|parameters)

Note that we **could** have priors that skew our posterior estimate in some direction. It would be entirely reasonable and possible!

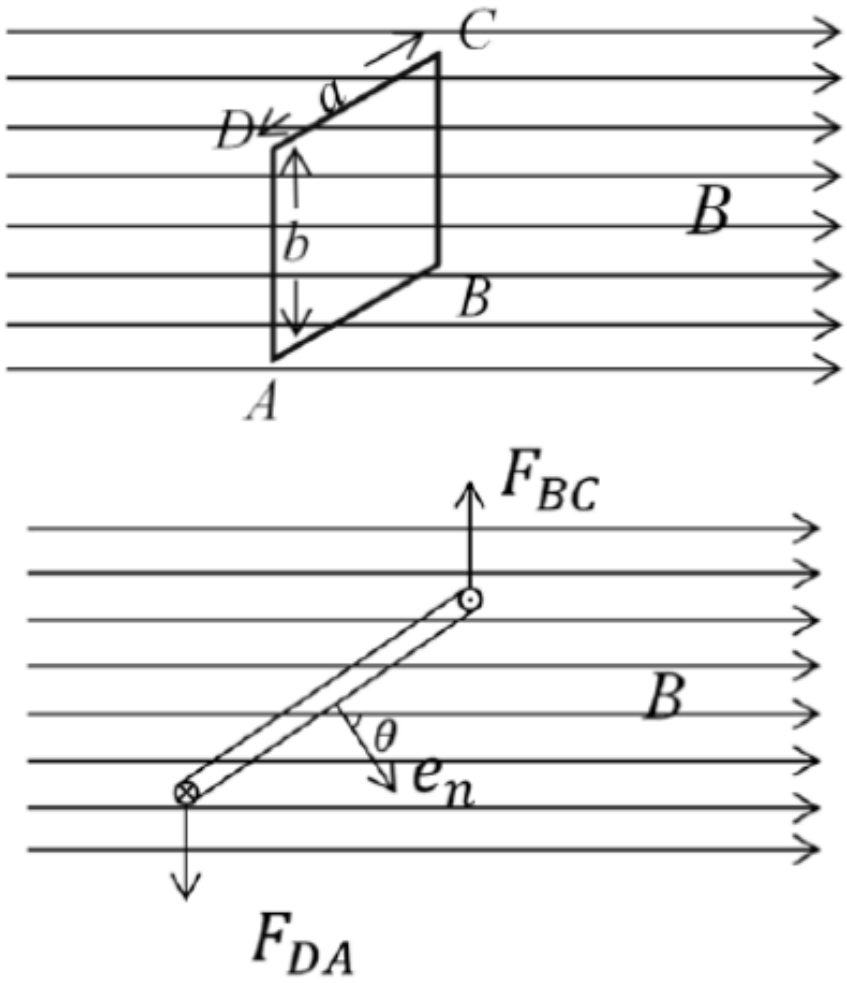
Getting back to our fits

Where are all these fits important
(to a particle physicist)?

Some physics

Following 2110.04673

For the CS folks here… it's OK, don't panic :)



Current I running through the loop. Biot-Savart law tells us that there is a force F on each of the b length arms, with magnitude bIB, and these are in opposite direction so there is a torque τ on the coil. Perpendicular distance between the two forces is $a \sin\theta$, so

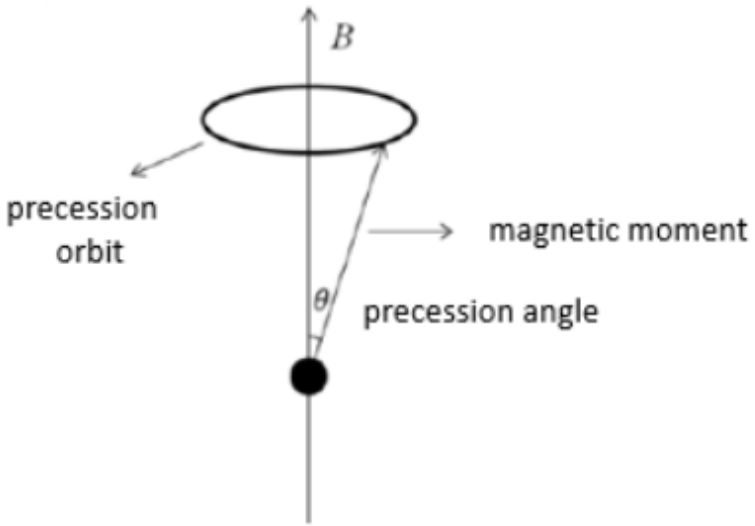
$$\tau = abIB \sin\theta = AIB \sin\theta$$

(A=ab)

$$|\tau| = IA\vec{e}_n \times \vec{B} = \vec{M} \times \vec{B}$$

$\vec{M} = IA\vec{e}_n$ with $\vec{e}_n$ the unit vector normal to the coil

How do we measure this in particle physics?



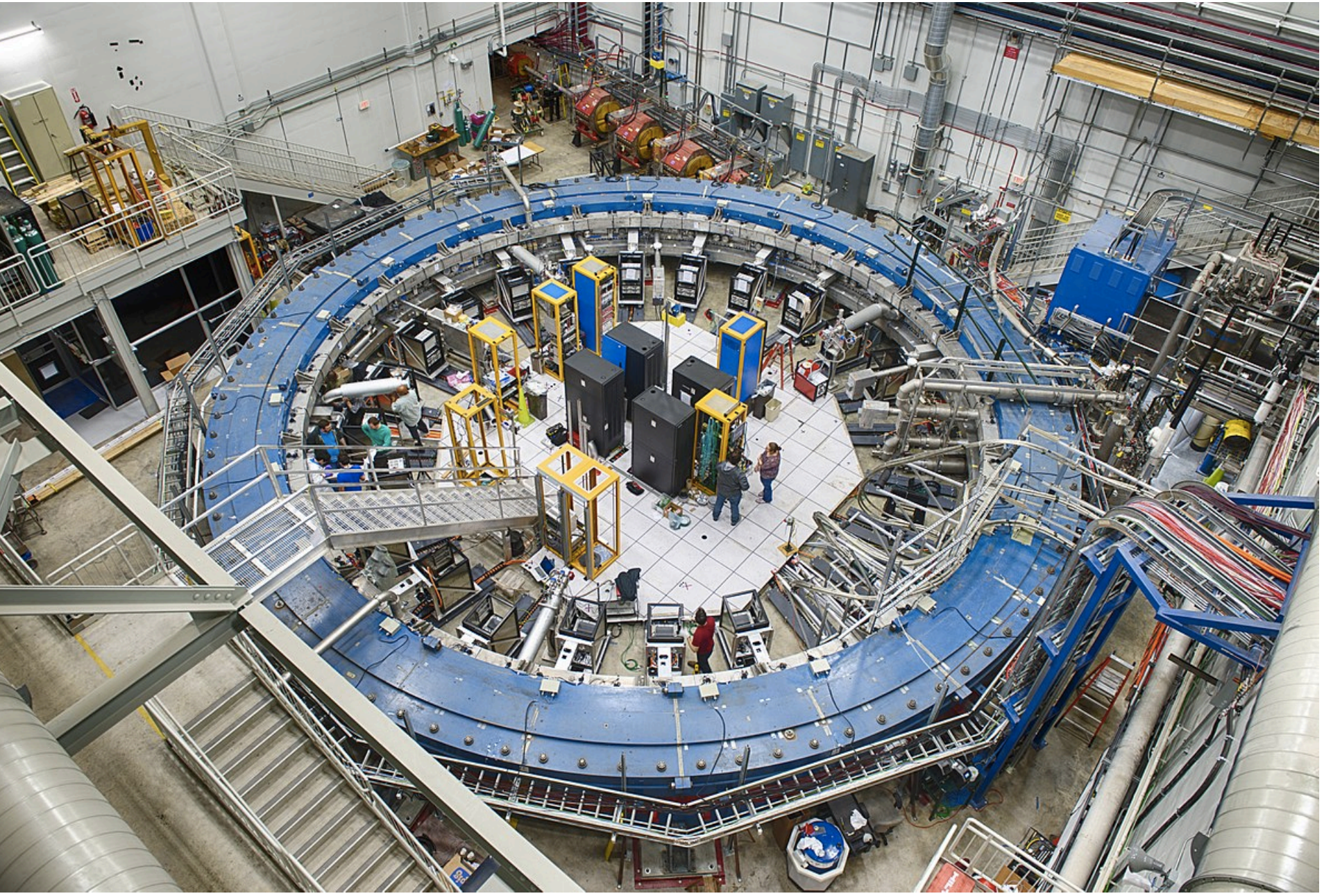
Analogous to a spinning top - spins rotates about the magnetic field

$$\frac{d\vec{S}}{dt} = \vec{L} = \vec{M} \times \vec{B} = g\frac{e}{2m}\vec{S} \times \vec{B}.$$

$$\omega = g\frac{e}{2m}B$$

Challenge - we are in the lab frame, not the rest frame of the muon!

g-2 at FNAL



How do we measure this, more fully?

Following 2110.04673

$$\frac{d\vec{S}}{dt} = (\vec{\omega}_c + \vec{\omega}_a) \times \vec{S},$$

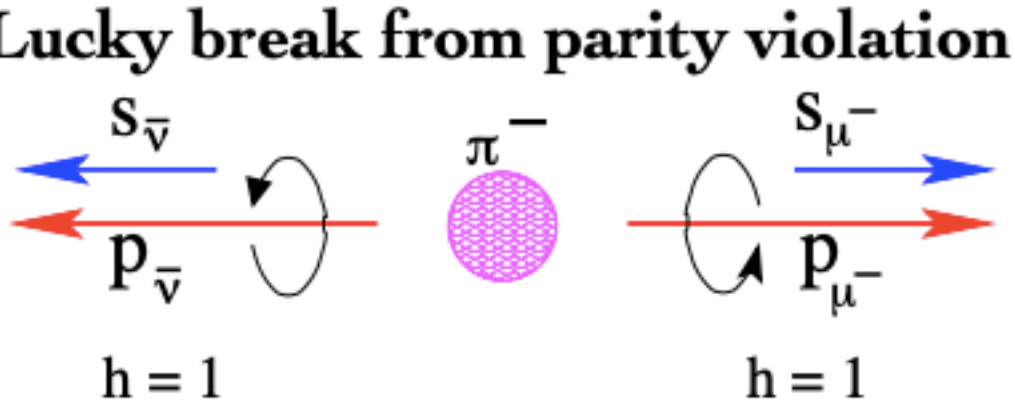
$$\vec{\omega}_c = -\frac{e}{\gamma m}\left(\vec{B} + \frac{\gamma^2}{\gamma^2 - 1}\frac{\vec{E} \times \vec{v}}{c^2}\right),$$

$$\vec{\omega}_a = -\frac{e}{m}\left[a\vec{B} - a\left(\frac{\gamma}{\gamma + 1}\frac{\vec{v} \cdot \vec{B}}{c^2}\right)\vec{v} + \left(a - \frac{1}{\gamma^2 - 1}\right)\frac{\vec{E} \times \vec{v}}{c^2}\right]$$

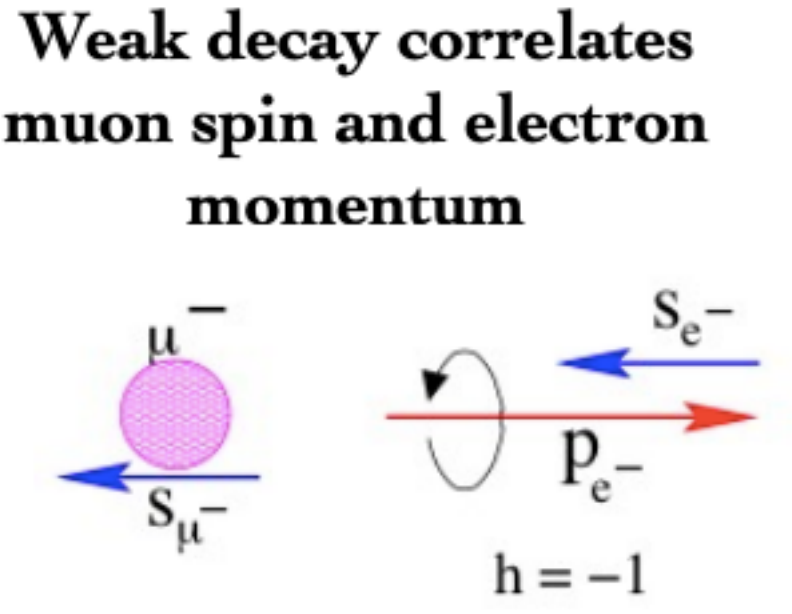
Pick the velocity giving "magic γ factor" to simplify ω_a. The other challenge is precisely measuring the **B** field and ensuring that it is uniform. Not easy! Use careful NMR probes and other tools

Muons come from pion decays. Thanks to parity violation, these are fully polarized!

https://indico.cern.ch/event/276476/contributions/1620139/attachments/501953/693162/g_minus_2_fewbuildins.pdf

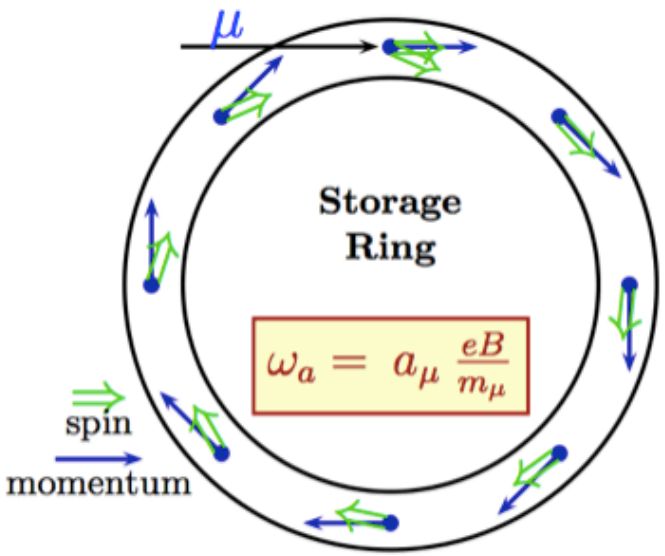


And there is a correlation between muon's spin and electron momentum from its decay



g-2 at FNAL

Spin precesses because $g_\mu \neq 2$

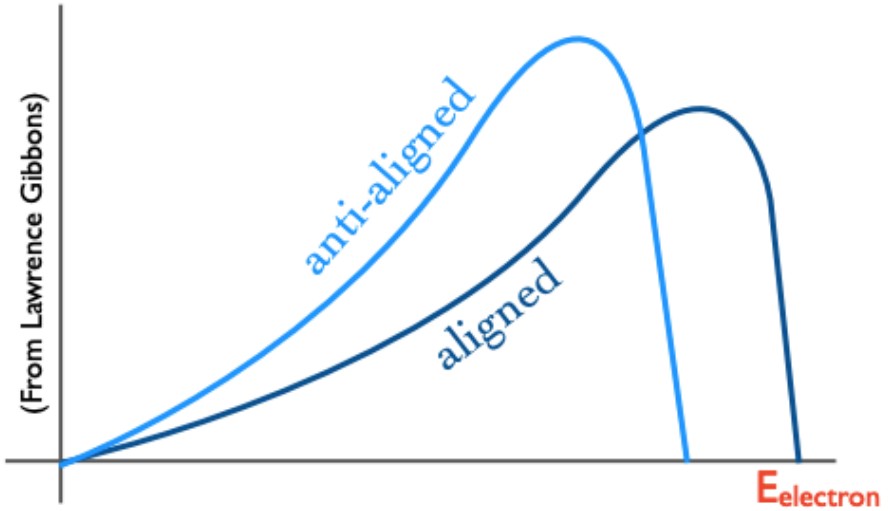


actual precession × 2

https://indico.cern.ch/event/276476/contributions/1620139/attachments/501953/693162/g_minus_2_fewbuildins.pdf

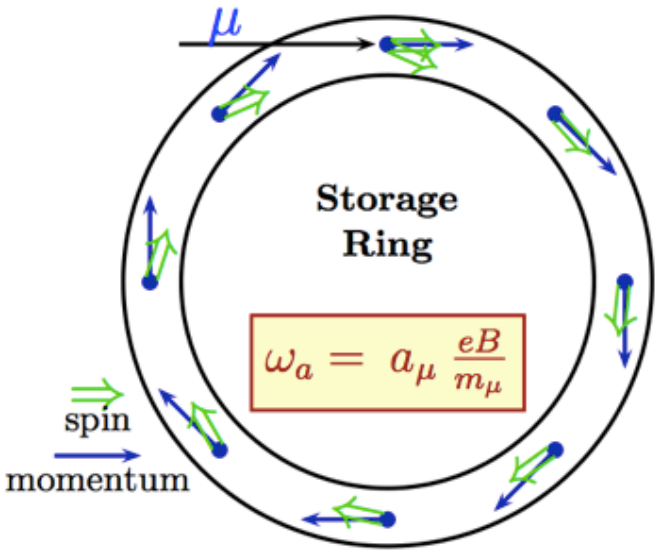
3. Measure Muon Spin Precession → 3. Measure Electron Energy

Harder electron spectrum when spin and momentum are aligned



g-2 at FNAL

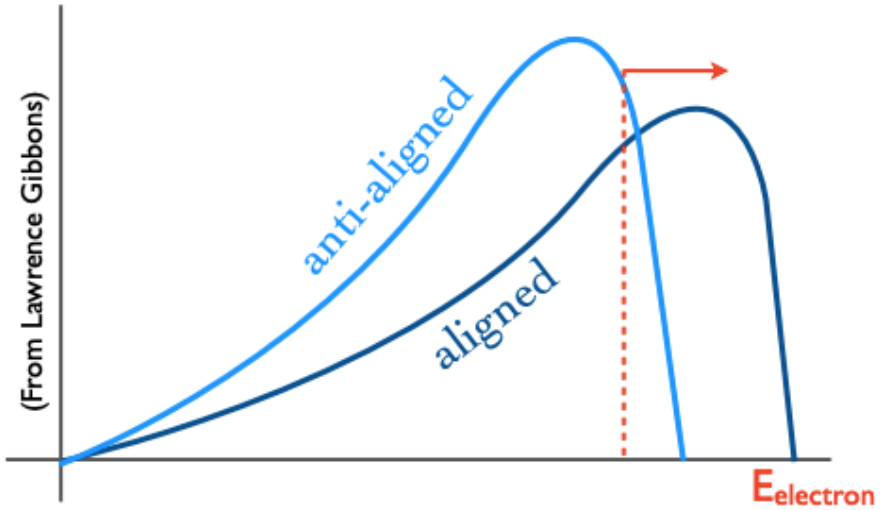
Spin precesses because $g_\mu \neq 2$



actual precession × 2

3. Measure Muon Spin Precession → 3. Measure Electron Energy

Harder electron spectrum when spin and momentum are aligned



Count N(e) above fixed threshold.
Oscillation rate ∝ a_μ

https://indico.cern.ch/event/276476/contributions/1620139/attachments/501953/693162/g_minus_2_fewbuildins.pdf

Not an easy thing to calculate at higher orders!

1805.01944

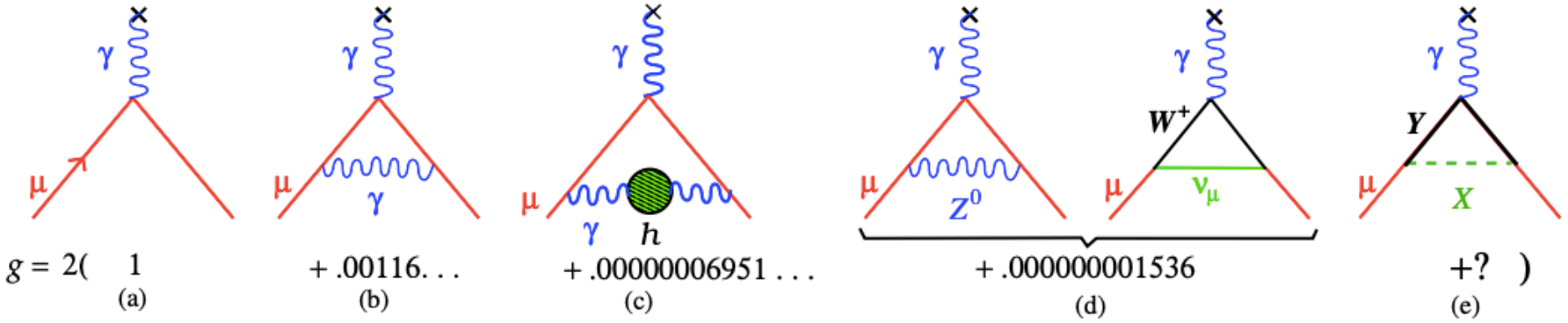


Figure 1: The Feynman graphs showing contributions to g for each of the Standard Model forces, ordered by size: (a) The Dirac interaction. (b) The lowest-order QED term $\alpha/2\pi$, which dominates the value of the anomaly. (c) The hadronic vacuum polarization contribution. (d) The lowest-order electroweak contributions. (The one-loop Higgs contribution is negligible.) (e) Potential contribution from new BSM particles X and Y.

Big challenge for evaluating the potential for BSM physics!
Measurement already accurate to 0.14 ppm!

What are the fit results?

https://journals.aps.org/prl/pdf/10.1103/7clf-sm2v

$$N(t) = N_0 \eta_N(t) e^{-t/\gamma\tau_\mu} \times \{1 + A\eta_A(t)\cos[\omega_a^m t + \varphi_0 + \eta_\phi(t)]\}$$

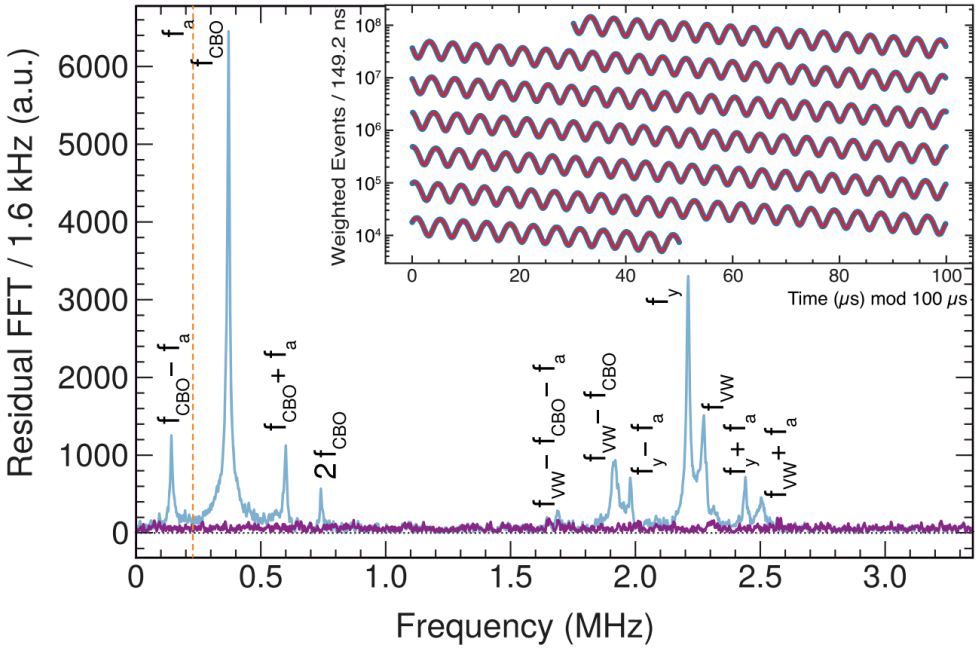


FIG. 1. Fourier transform of the residuals from the sum of the four fits to the Run-4/5/6 data excluding (blue) and including (purple) the $\xi(i)$ terms that incorporate the beam oscillation effects. The peaks correspond to the betatron frequencies (see [17] for frequency definitions). The rf-driven CBO damping has decreased the power at f_{CBO} compared to earlier data. The dashed line (orange) indicates the anomalous precession frequency f_a. Inset: the asymmetry-weighted e^+ time spectrum for the summed data (blue) and fit functions (red).

The number of measured decays oscillates as a cos term, but with an exponential term as muons decay!

We'll get to the Fourier transform part later

Following Eq. (2), we determine the muon anomaly

$$a_\mu(\text{Run-4/5/6}) = 1165920710(162)\times10^{-12}\ (139\text{ppb}),$$
$$a_\mu(\text{Run-1-6}) = 1165920705(148)\times10^{-12}\ (127\text{ppb}),$$

Higgs bosons can decay to photon pairs

https://arxiv.org/pdf/2308.07216

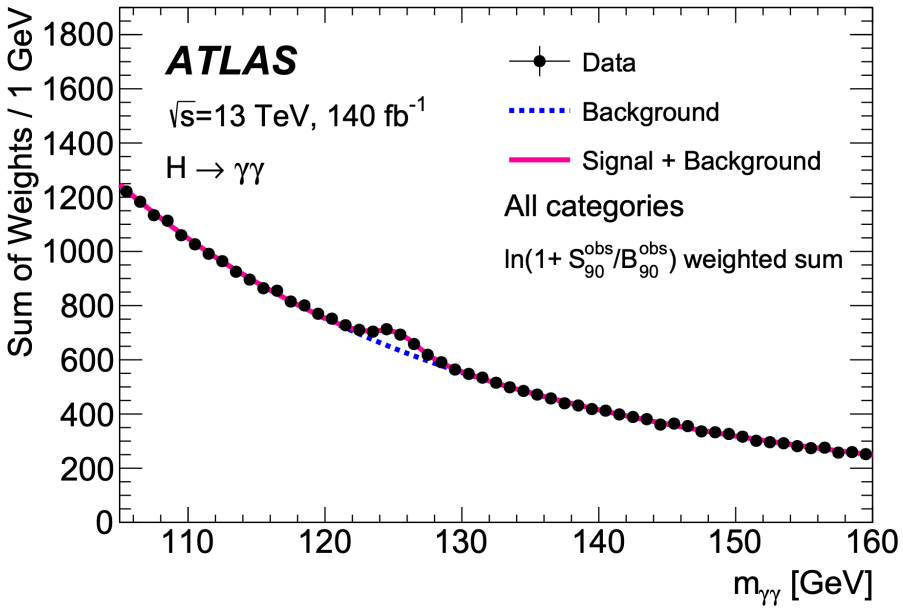
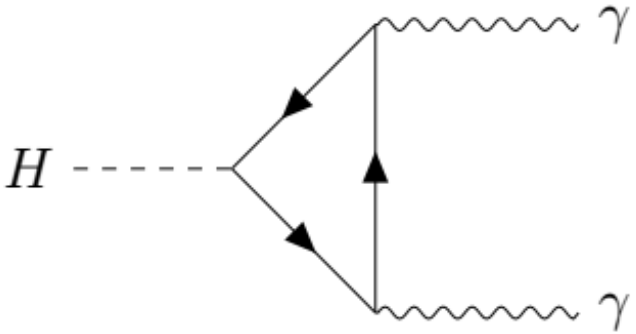


Figure 2: Diphoton invariant mass distribution of all selected data events (black dots with error bars), overlaid with the result of the fit (solid red line). For both the data and the fit, each category is weighted by a factor $\ln(1 + S_{90}^{\text{obs}}/B_{90}^{\text{obs}})$, where S_{90}^{obs} and B_{90}^{obs} are the fitted signal and background yields in the smallest $m_{\gamma\gamma}$ interval containing 90% of the expected signal. The dotted line describes the background component of the model.

Can discover/study/ measure the Higgs boson through diphoton decays. The background is a falling spectrum but… you need to accurately model the background in simulation and sidebands, or else you will not correctly measure the signal (or put the signal in the wrong place… or find signal that is not there!)

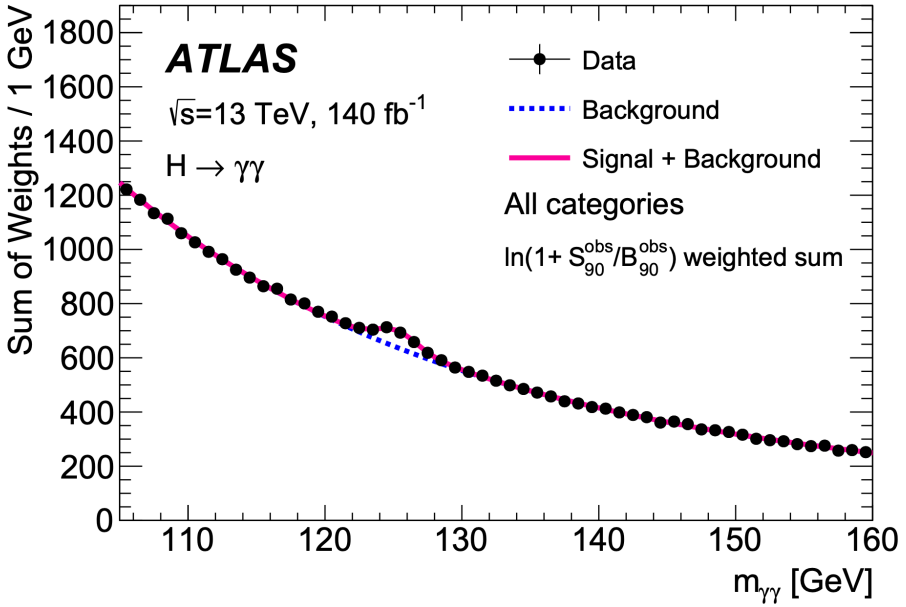


EM Algorithm

Let's imagine we want to fit some data to a combination of models, for example a signal and background model. This is exactly what we have here.

A priori, we do not know whether any given data measurement comes from signal or background (life would be easy if we could know that!). More generally, what if we want to fit some data where we have *latent variables* (ones that are not directly measured, such as m_{yy}, but are hidden and can only be inferred, such as the class an event belongs to)

Imagine we have k classes of data (here k=2, but can be larger), and we want to find $P(x_i \mid z_i, \theta)$ where x_i is the ith measurement in our dataset, θ are the parameters of our model, and z_i is the group that the ith measurement belongs to.



EM Algorithm

$P(x_i, z_i \mid \theta) = P(x_i \mid z_i, \theta)P(z_i)$ from Bayes theorem

In other words, the probability of a specific measurement occurring from a certain model given certain parameters is the probability that it occurred given that model time the probability of the model. Let's find the probabilities for any z_i

$$P(x_i \mid \theta) = \sum_{z_i=1}^{k} P(x_i, z_i \mid \theta) = \sum_{z_i=1}^{k} P(x_i \mid z_i, \theta)P(z_i)$$

So the total probability of our dataset given some model parameters is:

$$P(x_1, x_2 \ldots x_N \mid \theta) = \prod_{i=1}^{N} P(x_i \mid \theta) = \prod_{i=1}^{N} \sum_{z=1}^{k} P(x_i \mid z, \theta)P(z)$$

How do we maximize $P(x_1, x_2 \ldots x_N \mid \theta)$? We can do this numerically using Gradient descent or other minimization/maximization techniques, but this can be unstable (often our techniques will ping-pong back and forth around the best answer, or maybe not find the global minimum). How else can we do this?

EM Algorithm

So, let's imagine we have maximized θ. What are the values of z?

$$P(z \mid x_i, \theta) = \frac{P(x_i, z_i \mid \theta)}{P(x_i \mid \theta)} = \frac{P(x_i \mid z_i, \theta)P(z_i)}{\sum_z P(x_i \mid z, \theta)P(z)}$$

EM Algorithm

Remember that the total probability of our dataset given some model parameters is:

$$P(x_1, x_2 \ldots x_N | \theta) = \prod_{i=1}^{N} P(x_i | \theta) = \prod_{i=1}^{N} \sum_{z=1}^{k} P(x_i | z, \theta) P(z)$$

Let's take a sidebar to averages and inequalities.

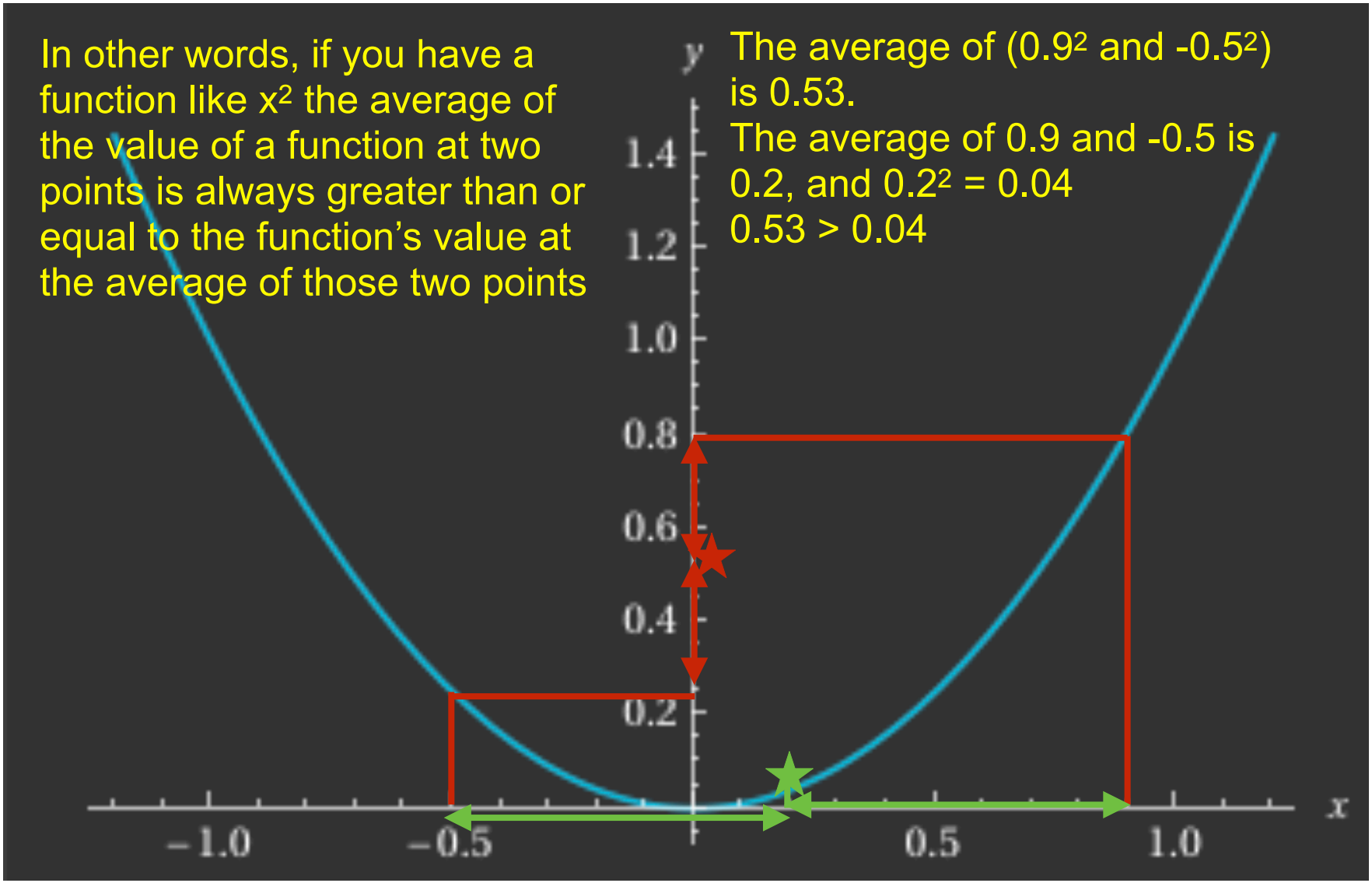
Imagine we have some positive quantities a_z (k of them) and we take a weighted average of them with positive weights $q_z \geq 0, \sum_{z=1}^{k} q_z = 1.$

Jensen's inequality tells us that the log of the average is greater than or equal to the average of the log:

$$\log \sum_{z=1}^{k} q_z a_z \geq \sum_{z=1}^{k} q_z \log a_z$$

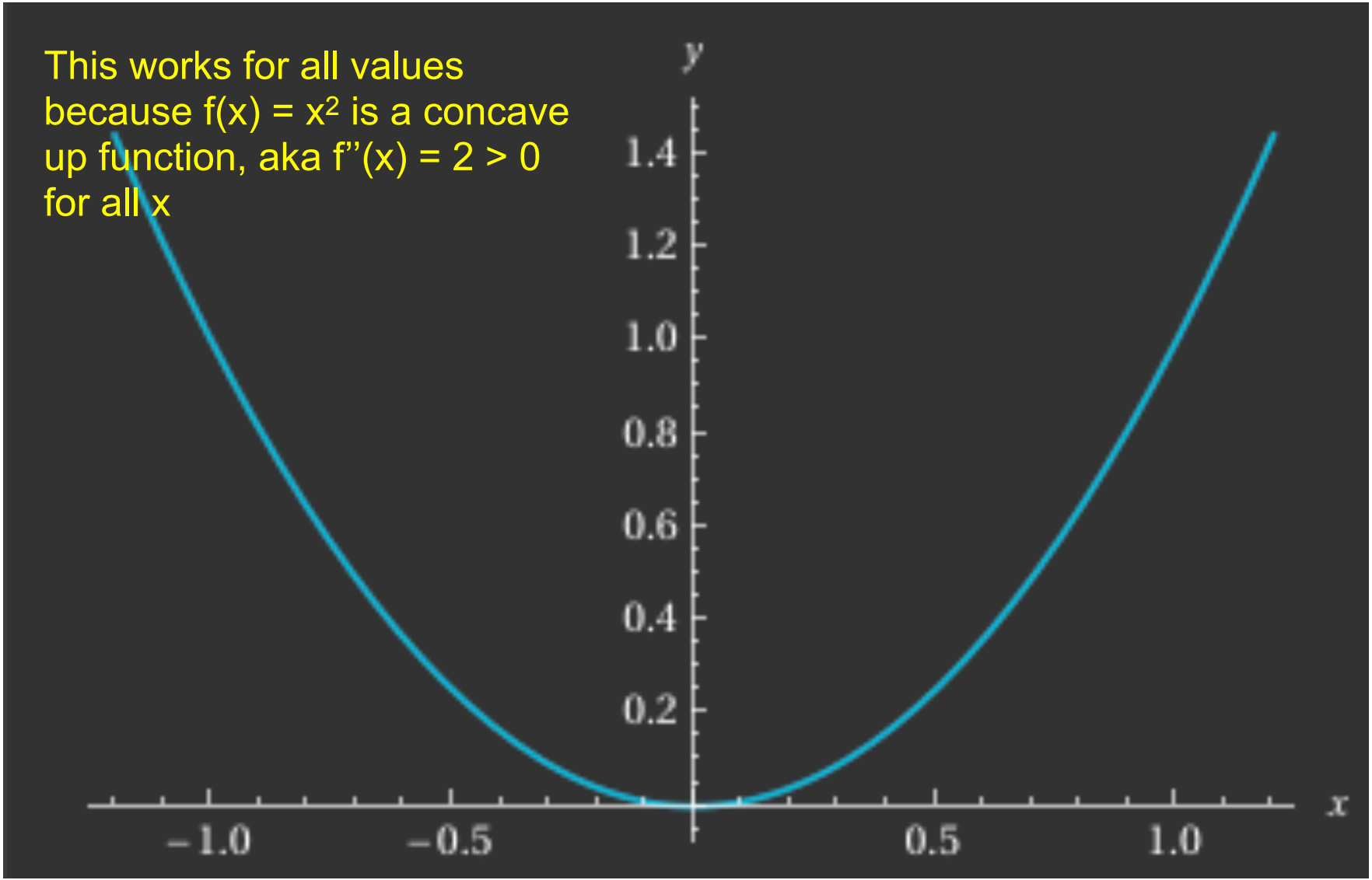
On Jensen's equality

The simplest way to think about this, recall that $E[x^2] >= (E[x])^2$



On Jensen's equality

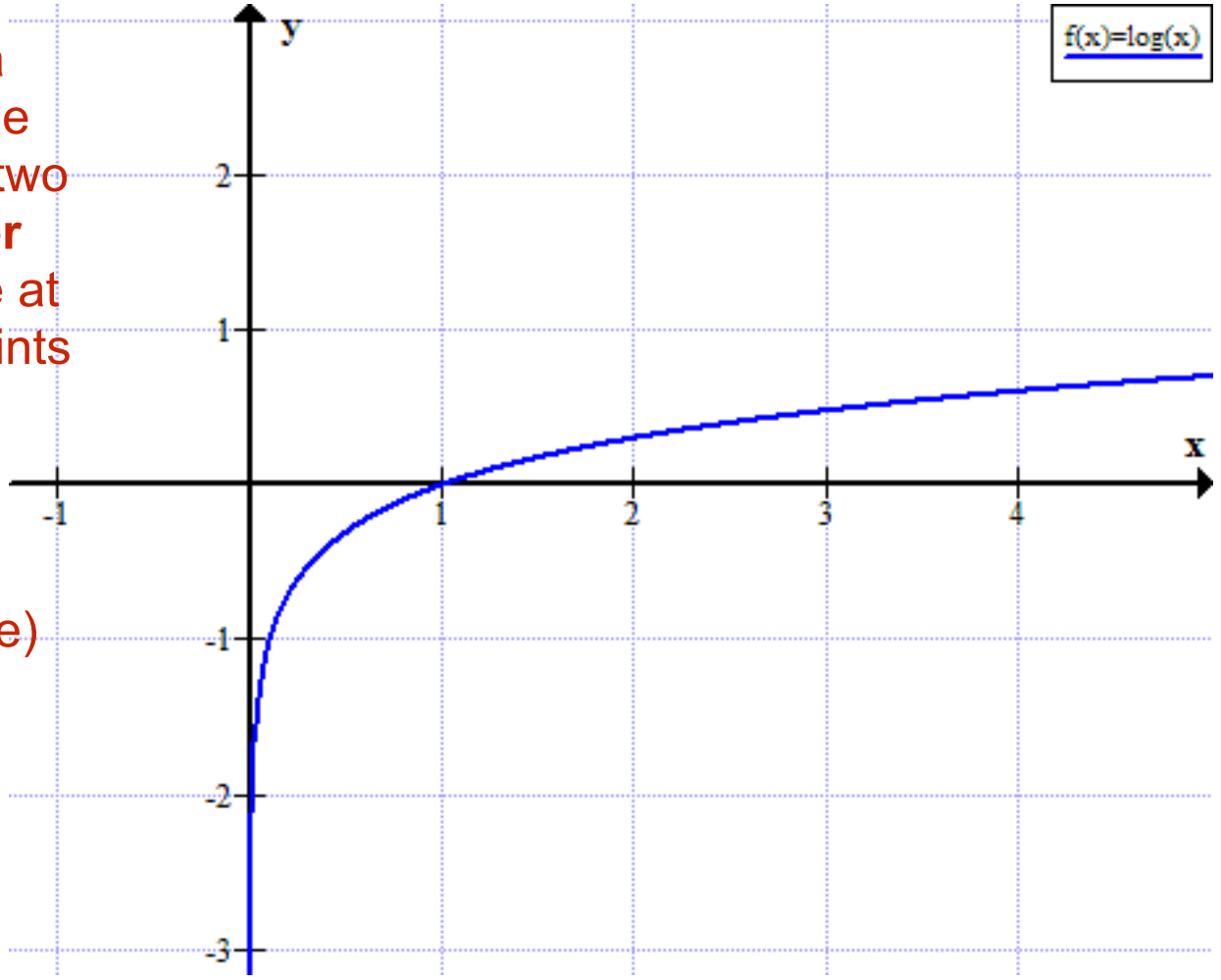
The simplest way to think about this, recall that $E[x^2] >= (E[x])^2$



On Jensen's equality

What about ln(x), aka $\log_e(x)$?

In other words, if you have a function like ln(x) the average of the value of a function at two points is always **less than or equal** to the function's value at the average of those two points



Ex:
The average of ln(1) and ln(e) is (0 + 1)/2 = 1/2 = 0.5
And ln([1+e]/2) = 0.62 > 0.5

Using Jensen's inequality

For some $q_z \geq 0, \sum_{z=1}^{k} q_z = 1$, Jensen's inequality tells us that the log of the average is greater than or equal to the average of the log: $\log \sum_{z=1}^{k} q_z a_z \geq \sum_{z=1}^{k} q_z \log a_z$

Remember that q_z are just some positive weights that sum to 1, and we can choose any of them that we want!

Replace a_z with p_z/q_z and we get

$$\log \sum_{z=1}^{k} p_z \geq \sum_{z=1}^{k} q_z \log \frac{p_z}{q_z}$$

Note that if we pick $q_z = \frac{p_z}{\sum_{z=1}^{k} p_z}$ and plug in to the right hand side we get

$\sum_{z=1}^{k} q_z \log \frac{p_z}{q_z} = \sum_{z=1}^{k} q_z \log \left(\sum_{z=1}^{k} p_z \right) = \log \sum_{z=1}^{k} p_z$ (since $\sum_{z=1}^{k} q_z = 1$) which is equality! So for this choice of q_z: $\log \sum_{z=1}^{k} p_z = \sum_{z=1}^{k} q_z \log \frac{p_z}{q_z}$

Using Jensen's inequality

Getting back to our problem, remember that the total probability of our dataset given some model parameters is: $P(x_1, x_2 \ldots x_N | \theta) = \prod_{i=1}^{N} P(x_i | \theta) = \prod_{i=1}^{N} \sum_{z=1}^{k} P(x_i | z, \theta) P(z)$

Following the book, let's call P(z) = γ_z so then

$P(x_1, x_2 \ldots x_N | \theta, \gamma) = \prod_{i=1}^{N} P(x_i | \theta, \gamma) = \prod_{i=1}^{N} \sum_{z=1}^{k} P(x_i | z, \theta) \gamma_z$ and we want to maximize this!

We can maximize the log of the probability instead:

$$\ln P(x_1, x_2 \ldots x_N | \theta, \gamma) = \ln \left(\prod_{i=1}^{N} P(x_i | \theta, \gamma) \right) = \ln \left(\prod_{i=1}^{N} \sum_{z=1}^{k} P(x_i | z, \theta) \gamma_z \right)$$

$$\ln P(x_1, x_2 \ldots x_N | \theta, \gamma) = \sum_{i=1}^{N} \ln \sum_{z=1}^{k} \gamma_z P(x_i | z, \theta).$$

Let's look at the ith term, $\ln \sum_{z=1}^{k} \gamma_z P(x_i | z, \theta)$. Jensen tells us that

$\ln \sum_{z=1}^{k} \gamma_z P(x_i | z, \theta) \geq q_z^i \ln \frac{\gamma_z P(x_i | z, \theta)}{q_z^i}$ where we can choose any q_z^i we want

Using Jensen's inequality

So we want to maximize $\ln P(x_1, x_2 \ldots x_N | \theta, \gamma) = \sum_{i=1}^{N} \ln \sum_{z=1}^{k} \gamma_z P(x_i | z, \theta)$.

And we know that

$\ln \sum_{z=1}^{k} \gamma_z P(x_i | z, \theta) \geq q_z^i \ln \frac{\gamma_z P(x_i | z, \theta)}{q_z^i}$ where we can choose any q_z^i we want. So

$\ln P(x_1, x_2 \ldots x_N | \theta, \gamma) \geq \sum_{i=1}^{N} \sum_{z=1}^{k} q_z^i \ln \frac{\gamma_z P(x_i | z, \theta)}{q_z^i}$ and we can change the "greater than or equal" to "is exactly equal to" when

$q_z^i = \frac{\gamma_z P(x_i | z, \theta)}{\sum_z \gamma_z P(x_i | z, \theta)}$. So we if use this value in our weighted average then we maximize the probability of our weighted average! This choice will maximize the right-hand side, but what about our choice of γ, θ? We need to do a double maximization and maximize both.

EM algorithm: Maximize with respect to q_z^i holding the parameters γ, θ constant, then hold q_z^i constant and maximize with respect to γ, θ. To maximize with respect to any one of the parameters θ we take the partial derivative:

$\sum_{i=1}^{N} \sum_{z=1}^{k} q_z^i \frac{\partial \ln P(x_i | z, \theta)}{\partial \theta}$ = 0 (since q_z^i, γ do not depend on θ).

What about maximizing with respect to γ?

$\ln P(x_1, x_2 \ldots x_N | \theta, \gamma) \geq \sum_{i=1}^{N} \sum_{z=1}^{k} q_z^i \ln \frac{P(x_i | z, \theta)\gamma_z}{q_z^i}$ is maximized with

$q_z^i = \frac{\gamma_z P(x_i | z, \theta)}{\sum_z \gamma_z P(x_i | z, \theta)}$. To maximize with respect to any one of the parameters θ:

$\sum_{i=1}^{N} \sum_{z=1}^{k} q_z^i \frac{\partial \ln P(x_i | z, \theta)}{\partial \theta}$ = 0. What about the γ_z? Remember that P(z) = γ_z so $\sum_{z=1}^{k} \gamma_z = 1$.

Use a Lagrange multiplier λ to enforce the constraint:

$$\frac{\partial}{\partial \gamma_z} \left[\sum_{i=1}^{N} \sum_{z=1}^{k} q_z^i \ln \frac{P(x_i | z, \theta)\gamma_z}{q_z^i} + \lambda(1 - \sum_{z=1}^{k} \gamma_z) \right] = 0$$

$$\sum_{i=1}^{N} q_z^i \frac{q_z^i}{P(x_i | z, \theta)\gamma_z} \frac{P(x_i | z, \theta)}{q_z^i} - \lambda = 0 \text{ and thus } \sum_{i=1}^{N} \frac{q_z^i}{\gamma_z} - \lambda = 0$$

So we have $\gamma_z = \frac{1}{\lambda} \sum_{i=1}^{N} q_z^i$ and remembering that $\sum_{z=1}^{k} \gamma_z = 1$, $\sum_{z=1}^{k} \gamma_z = \frac{1}{\lambda} \sum_{z=1}^{k} \sum_{i=1}^{N} q_z^i = 1$

What about maximizing with respect to γ?

We found $\gamma_z = \frac{1}{\lambda}\sum_{i=1}^{N} q_z^i$ and also

$\sum_{z=1}^{k}\gamma_z = \frac{1}{\lambda}\sum_{z=1}^{k}\sum_{i=1}^{N} q_z^i = \frac{1}{\lambda}\sum_{i=1}^{N}\sum_{z=1}^{k} q_z^i = 1$ but also we know that $\sum_{z=1}^{k} q_z^i = 1$ (these are just the weights we used, which must sum to 1). So

$\frac{1}{\lambda}\sum_{i=1}^{N}\sum_{z=1}^{k} q_z^i = 1 = \frac{1}{\lambda}\sum_{i=1}^{N} 1 = \frac{N}{\lambda}$ so $\lambda = N$ and $\gamma_z = \frac{1}{N}\sum_{i=1}^{N} q_z^i$. PHEW

$$q_z^i = \frac{\gamma_z P(x_i \mid z, \theta)}{\sum_z \gamma_z P(x_i \mid z, \theta)} \qquad \gamma_z = \frac{1}{N}\sum_{i=1}^{N} q_z^i \qquad \sum_{i=1}^{N}\sum_{z=1}^{k} q_z^i \frac{\partial \ln P(x_i \mid z, \theta)}{\partial \theta} = 0$$

The EM Algorithm is now ready

Pick some values for our parameters (ideally not crazy guesses). Then for all i (index corresponding to the N events) and all z (index corresponding to the k hidden values such as the k weights) calculate:

1. $q_z^i = \frac{\gamma_z P(x_i \mid z, \theta)}{\sum_z \gamma_z P(x_i \mid z, \theta)}$
2. Solve for the new values of the parameters using $\sum_{i=1}^{N} \sum_{z=1}^{k} q_z^i \frac{\partial \ln P(x_i \mid z, \theta)}{\partial \theta} = 0$ and $\gamma_z = \frac{1}{N} \sum_{i=1}^{N} q_z^i$
3. Repeat above two steps until we converge. Note that step 2 can be done analytically in some cases (as we'll do in the following) or using recipes from future chapters of this course

EM algorithm - first do the Expectation step (calculate the expected values of the missing or latent variables) and then Maximize the log likelihood. And repeat.

EM algorithm for a mixture of two Gaussians

For a Gaussian, we have: $P(x_i \mid z, \theta) = \frac{1}{\sqrt{2\pi\sigma_z^2}} e^{-\frac{(x_i - \mu_z)^2}{2\sigma_z^2}}$ and then

$$\ln P(x_i \mid z, \theta) = \frac{-1}{2} \ln(2\pi\sigma_z^2) - \frac{(x_i - \mu_z)^2}{2\sigma_z^2}$$

Plug into $\sum_{i=1}^{N} \sum_{z=1}^{k} q_z^i \frac{\partial \ln P(x_i \mid z, \theta)}{\partial \mu} = 0$

$$\sum_{i=1}^{N} q_z^i \frac{(x_i - \mu_z)}{\sigma_z^2} = 0$$

Plug into $\sum_{i=1}^{N} \sum_{z=1}^{k} q_z^i \frac{\partial \ln P(x_i \mid z, \theta)}{\partial \sigma_z} = 0$

$$\sum_{i=1}^{N} q_z^i \left[\left(-\frac{1}{\sigma_z} + \frac{(x_i - \mu_z)^2}{\sigma_z^3} \right] = 0 \right.$$

EM algorithm for a mixture of two Gaussians

$$\sum_{i=1}^{N} q_z^i \frac{(x_i - \mu_z)}{\sigma_z^2} = 0$$

$$\sum_{i=1}^{N} q_z^i \left[\left(-\frac{1}{\sigma_z} + \frac{(x_i - \mu_z)^2}{\sigma_z^3} \right] \right. = 0$$

$$\sum_{i=1}^{N} q_z^i (x_i - \mu_z) = 0$$

$$\sum_{i=1}^{N} q_z^i \left[-\sigma_z^2 + (x_i - \mu_z)^2 \right] = 0$$

$$\sum_{i=1}^{N} q_z^i x_i = \mu_z \sum_{i=1}^{N} q_z^i$$

$$\sum_{i=1}^{N} q_z^i \sigma_z^2 = \sum_{i=1}^{N} q_z^i (x_i - \mu_z)^2$$

$$\mu_z = \frac{\sum_{i=1}^{N} q_z^i x_i}{\sum_{i=1}^{N} q_z^i}$$

These analytically maximize the log likelihood (M step) for each choice of E step on the latent variables

$$\sigma_z^2 = \frac{\sum_{i=1}^{N} q_z^i (x_i - \mu_z)^2}{\sum_{i=1}^{N} q_z^i}$$

Exercise 11.4

Have data of exoplanet showing distance to their sun (in units of distance from Earth to sun), assume there are two classes of these planets. Fit a mixture of two Gaussians to the data and for the first 20 planets find the relative probabilities that they belong to each class

Good job, Generative AI

```
# Exercise 11.4
from math import sqrt,pi
from numpy import empty, zeros, array, loadtxt, linspace
from numpy import copy, exp, genfromtxt
import matplotlib.pyplot as plt
from google.colab import files

k = 2 ### number of groups
xmin = 0.0
xmax = 20.0
npoints = 400
xpoints = linspace(xmin,xmax,npoints+1)
target = 1e-6 ### for everything, could have separate ones
Nloopmax = 100

# Gaussian
def gaussian(x,mu,sigma):
  return exp(-(x-mu)**2/(2*sigma**2))/(sqrt(2*pi)*sigma)

# Read the data
uploaded = files.upload() ### To upload!
data = genfromtxt('exoplanets.txt')
x = data[:,4]
N = len(x)

# Choose initial values for the parameters, these do not have to be perfect
mu = array([1.0,3.0],float)
sigma = array([1.0,1.0],float)
gamma = array([0.5,0.5],float)
```

Exercise 11.4


```
# Main loop
q = empty([N,k],float)
for j in range(Nloopmax): ### need to make sure we stop before then, we'll check at the end
    # Calculate the qiz's
    for i in range(N):
      for z in range(k):
        q[i,z] = gamma[z]*gaussian(x[i],mu[z],sigma[z])
      norm = sum(q[i])
      q[i] /= norm

    # Calculate updated values for the parameters, we can use the analytic form here
    oldmu = copy(mu)
    for z in range(k):
      norm = sum(q[:,z])
      mu[z] = sum(q[:,z]*x)/norm
      sigma[z] = sqrt(sum(q[:,z]*(x-mu[z])**2)/norm)
      gamma[z] = norm/N

    # Check for convergence
    delta = max(abs(mu-oldmu))
    if delta < target:
      break

    if (j == Nloopmax - 1):
      print("WARNING! Made it up to the max Nloop and did not converge")
```

The E step (expected values of the latent variables)

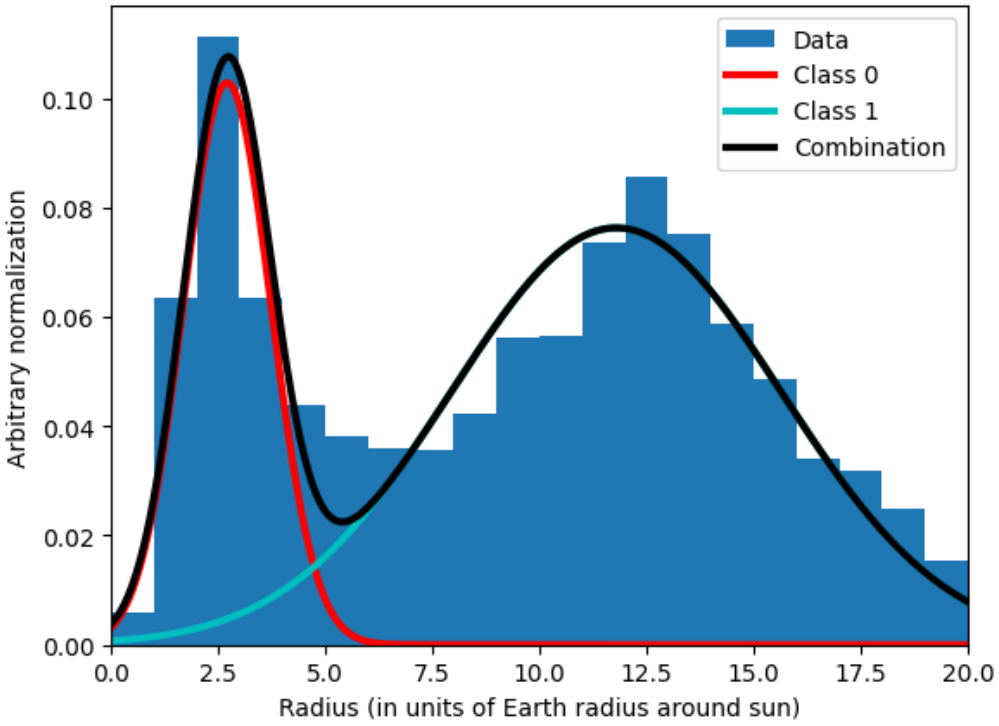
The M step (maximize the likelihood given those latent variables)

Just checking mu, could check all parameters before deciding to stop or not

Exercise 11.4


```
# Make the plot
plt.hist(x,bins=20,range=[xmin,xmax],density=True, label="Data")
fitsum = zeros(npoints+1,float)
colors = ["r","c"]
for z in range(k):
  fit = gamma[z]*exp(-(xpoints-mu[z])**2 / (2*sigma[z]**2))/sqrt(2*pi*sigma[z]**2)
  fitsum += fit
  plt.plot(xpoints,fit,colors[z], linewidth=3, label=f"Class {z}")
plt.plot(xpoints, fitsum, "k", linewidth=3, label="Combination")
plt.legend()
plt.xlabel("Radius (in units of Earth radius around sun)")
plt.ylabel("Arbitrary normalization")
plt.xlim(xmin,xmax)
plt.show()

Ntocheck = 20
for i in range(Ntocheck):
  r = x[i]
  fit0 = gamma[0]*exp(-(r-mu[0])**2 / (2*sigma[0]**2))/sqrt(2*pi*sigma[0]**2)
  fit1 = gamma[1]*exp(-(r-mu[1])**2 / (2*sigma[1]**2))/sqrt(2*pi*sigma[1]**2)
  print("r = ",r,"and fit0 = ",fit0,"and fit1 = ",fit1, "and max ratio = ", max(fit0/fit1,fit1/fit0))
```

Exercise 11.4 Results



Model is quite certain about classification for almost all planets. It only has trouble for radii ~ 5, where the two curves cross

```
r =  5.8181633 and fit0 =  0.001009188284179668 and fit1 =  0.022921942280485406 and max ratio =  22.713246516845775
r =  11.2154 and fit0 =  9.515048723355067e-17 and fit1 =  0.07541666710033923 and max ratio =  792604108429061.5
r =  11.3113 and fit0 =  4.340187357377608e-17 and fit1 =  0.07567471781344243 and max ratio =  1743581822218061.5
r =  6.54449 and fit0 =  9.036658022725918e-05 and fit1 =  0.030163839429374065 and max ratio =  333.79418977144286
r =  8.6950898 and fit0 =  3.693757152507361e-09 and fit1 =  0.055210289016201584 and max ratio =  14946919.01407873
r =  14.7752 and fit0 =  5.780635087701048e-32 and fit1 =  0.056512281067575075 and max ratio =  9.776137087049712e+29
r =  3.19621 and fit0 =  0.09179984093680041 and fit1 =  0.006329876852915632 and max ratio =  14.502626681357329
r =  13.1874503 and fit0 =  1.5828703861118204e-24 and fit1 =  0.07142438260381855 and max ratio =  4.512332988885221e+22
r =  5.9115908 and fit0 =  0.0007611299507210051 and fit1 =  0.023793245667351897 and max ratio =  31.260424904857537
r =  1.44656 and fit0 =  0.04794078186300099 and fit1 =  0.0020728595052549276 and max ratio =  23.12784910963132
r =  14.0608342 and fit0 =  1.7294508929262759e-28 and fit1 =  0.06412526164525509 and max ratio =  3.707839401947602e+26
r =  15.4706 and fit0 =  1.494937362611215e-35 and fit1 =  0.04834725823588883 and max ratio =  3.234065817409261e+33
r =  3.34688 and fit0 =  0.08465342183153045 and fit1 =  0.006901714830002936 and max ratio =  12.265563547124192
r =  3.4262 and fit0 =  0.08041263595658586 and fit1 =  0.007218790850234177 and max ratio =  11.139349736663625
r =  3.7255381 and fit0 =  0.06273939649754075 and fit1 =  0.008519685286725205 and max ratio =  7.364050946259367
r =  2.87648 and fit0 =  0.1014569013707609 and fit1 =  0.005241877964487321 and max ratio =  19.355067412502006
r =  2.76586 and fit0 =  0.10266422240787523 and fit1 =  0.004902886819937772 and max ratio =  20.939545655100044
r =  2.52314 and fit0 =  0.10112802310730992 and fit1 =  0.004221717509625799 and max ratio =  23.9542373161471
r =  1.20704 and fit0 =  0.03492071090493188 and fit1 =  0.0017507702520882002 and max ratio =  19.945912870795482
r =  14.581841 and fit0 =  5.2935459957455865e-31 and fit1 =  0.058677663973470094 and max ratio =  1.1084755666736291e+29
```

Homework #2 (page 1/3) https://classroom.github.com/a/yhBpCtBU

1) Write **pseudocode** to take a set of N (x,y) points, each with associated errors, and return the best fit straight line, errors on parameters, and chi2/ndf. Include any error checking that might be needed
2) Read the data (Table 1) from Hubble's original paper and plot it (velocity moving away from us as a function of distance from us)
3) Write actual code to determine the best-fit line and plot that on the same axis, assuming constant errors. How does your value for Hubble's constant (the slope) compare with current best estimates of ~70 (km/s)/Mpc? Plot that current best estimate on the same axis. Add legends and axis labels
4) Do the same for Exercise 11.1 in the textbook. Then redo the problem assuming a constant fractional error of 2% on the y values in the data. Give the chi2/ndf and errors on a,b and h in this new version. How do your values of a,b and h change? Then redo this again assuming a constant fractional error of 1% on the values in the data added in quadrature with a 0.03V error. How do your chi2/ndf values change? How do the errors on your parameters change? Providing proper errors on your data is important!

Homework #2 (continued)

5) For PHYS510 only: On the course git page you will find a comma-separated file containing several thousand entries for baseball players over the past 5 years. Plot the batting average (the fraction of the time a batter succeeds) as a function of average launch angle (angle at which the ball leaves the baseball bat with respect to the horizontal). Fit this data to a second order polynomial and show your fit on the data. You can use numerical recipes you find on the web or elsewhere for this. What does your 2nd order fit show as the best average angle to succeed (ie to have a hit)? And what does it show as the drop in expected average if a ball is hit with a shift of 10 degrees from optimal?

   If you are using python you may want to investigate the csv package. If you want to use colab, you will need to figure out how to get the data into colab. I suggest uploading to your private google drive:

```
from google.colab import drive
drive.mount('/content/drive')
!ls "/content/drive/My Drive/baseball_stats.csv” ## to change if file moves
f = open('/content/drive/My Drive//baseball_stats.csv”) # something like this
```

Homework #2 (continued)

6) For PHYS510 only: On the course git page you will find a comma-separated file containing the measured speed of sound vs temperature over a very wide range of temperatures (albeit in funny non-SI units). These come from: https://www.engineeringtoolbox.com/air-speed-sound-d_603.html

Plot the temperature vs sound and fit the data to both 1st and 2nd order polynomials. You can use numerical recipes you find on the web or elsewhere for this. Comment on which model does a better job? How do the predictions compare at the very highest temperature?

Then fit to a 9th degree polynomial. What do you see? Does that make sense?

(In fact, the speed of sound should go as $\sqrt{1+T}$, and a Taylor expansion means that a linear approximation is valid for small ranges in temperature, and only stops working over very wide ranges such as in this problem)